

U-Space

Gestão de Espaços Acadêmicos da Universidade de São Paulo

Felipe Pinho
Gabriel Sanches
Kadu Barrille
Marcos O. Salvador

Objetivo da Aplicação

- **Organizar e divulgar espaços acadêmicos** para aulas e monitorias.
- Gerenciar **reservas de salas** e evitar conflitos de uso.
- Facilitar **comunicação e interação** entre alunos, monitores, docentes e administração.
- Permitir o aluno visualizar sua própria **grade horária**
- Criar uma **rede de integração acadêmica** completa.



Modelo Relacional

Principais Tabelas

Tabela Usuário

Criação da Tabela

```
CREATE TABLE usuario (  
  email VARCHAR(150) UNIQUE NOT NULL,  
  senha VARCHAR(50) NOT NULL,  
  nusp INTEGER PRIMARY KEY,  
  rg VARCHAR(9) UNIQUE,  
  foto BYTEA,  
  nome VARCHAR(100),  
  pnome VARCHAR(50),  
  unome VARCHAR(50)  
);
```

Função da Tabela

- População do Banco de Dados

Mais de 350 Usuários no BD

- Alunos
- Professores
- Funcionários

Vizualização dos Dados

SELECT * FROM usuario ORDER BY RANDOM() LIMIT 5;

email character varying (150)	senha character varying (50)	nusp [PK] integer	rg character varying (9)
prof_alves@usp.br	fDP5qKUthK	64433	911021091
funcionario_larissafaria@usp...	VISoz0SFdo	48	685555623
prof_ribeiro@usp.br	cJluatY3HO	69340	232458733
fernanda_martins@usp.br	ii1ChrZE	60495880	616477920
bruna_carvalho@usp.br	C3CwQe!1	61624345	876740782

foto bytea	nome character varying (100)	pnome character varying (50)	unome character varying (50)
[null]	Sabrina Alves	Sabrina	Alves
[null]	Larissa Faria	Larissa	Faria
[null]	Patrícia Ribeiro	Patrícia	Ribeiro
[null]	Fernanda Martins	Fernanda	Martins
[null]	Bruna Carvalho	Bruna	Carvalho

- Diferenciação por E-mail e Nusp

Principais Tabelas

Tabela Turma

Criação da Tabela

```
CREATE TABLE turma (  
  sigla_disciplina VARCHAR(20) NOT NULL,  
  codigo_turma VARCHAR(5) NOT NULL,  
  vigencia VARCHAR(10) NOT NULL,  
  ano INTEGER CHECK (ano BETWEEN 1980 AND 2025),  
  semestre SMALLINT CHECK(semestre in (1,2)),  
  PRIMARY KEY (sigla_disciplina, codigo_turma, vigencia),  
  FOREIGN KEY (sigla_disciplina) REFERENCES disciplina(sigla)  
    ON DELETE CASCADE ON UPDATE CASCADE  
);
```

Função da Tabela

- Materializar o oferecimento de uma disciplina

131 Turmas Inseridas

- 65 Turmas do Primeiro Semestre (Finalizadas)
- 66 Turmas do Segundo Semestre (Pendentes)

Chave Primária Tripla

- Disciplina, Código da Turma e Vigência

Vizualização dos Dados

SELECT * FROM turma ORDER BY RANDOM() LIMIT 4;

sigla_disciplina [PK] character varying (20)	codigo_turma [PK] character varying (5)
AUP2422	T01
EAD0610	T02
EAC0106	T01
4302112	T01
MAE0127	T01

vigencia [PK] character varying (10)	ano integer	semestre smallint
2025-1	2025	1
2025-1	2025	1
2025-1	2025	1
2025-2	2025	2
2025-1	2025	1

Principais Tabelas

Tabela Instituto

Criação da Tabela

```
CREATE TABLE instituto (  
    id_instituto SERIAL PRIMARY KEY,  
    sigla VARCHAR(10) UNIQUE,  
    contato VARCHAR(120) UNIQUE,  
    email VARCHAR(150),  
    endereco VARCHAR(100) UNIQUE,  
    logradouro VARCHAR(40),  
    telefone VARCHAR(20),  
    cep VARCHAR(8)  
);
```

Identificação

- Separados por IDs

7 Institutos Inseridos

- Contemplando IME e seus vizinhos
- **FAU, FEA, IO, IAG, IF e ECA**

Relação Mãe

- Controla vínculos de trabalho
- Localiza salas

Tabela Departamento

Criação da Tabela

```
CREATE TABLE departamento (  
    id_departamento SERIAL PRIMARY KEY,  
    nome VARCHAR(100) UNIQUE,  
    sigla VARCHAR(20) UNIQUE,  
    contato VARCHAR(50) UNIQUE,  
    email VARCHAR(150),  
    telefone VARCHAR(20),  
    id_instituto INTEGER,  
    FOREIGN KEY (id_instituto) REFERENCES instituto(id_instituto)  
        ON DELETE SET NULL ON UPDATE CASCADE  
);
```

Identificação

- Separados por IDs

19 Departamentos Inseridas

- Média de 3 departamentos por Instituto
- Coordenados por professores (Relação coordenação)

Controle de disciplinas

- Toda disciplina é retratada por um departamento

Principais Tabelas

Tabela Participa_em Criação da Tabela

```
CREATE TABLE participa_em (  
  nusp_aluno INTEGER NOT NULL,  
  media_final NUMERIC(5,2) CHECK (media_final BETWEEN 0 AND 10),  
  status SMALLINT CHECK(status in (1,2)), -- 1 Ativo, 2 Desativado  
  frequencia NUMERIC(5,2) CHECK (frequencia BETWEEN 0 AND 100),  
  sigla_disciplina VARCHAR(20) NOT NULL,  
  codigo_turma VARCHAR(5) NOT NULL,  
  vigencia VARCHAR(10) NOT NULL,  
  PRIMARY KEY (nusp_aluno, sigla_disciplina, codigo_turma, vigencia),  
  FOREIGN KEY (nusp_aluno) REFERENCES aluno(nusp_usuario)  
    ON DELETE CASCADE ON UPDATE CASCADE,  
  FOREIGN KEY (sigla_disciplina, codigo_turma, vigencia)  
    REFERENCES turma(sigla_disciplina, codigo_turma, vigencia)  
    ON DELETE CASCADE ON UPDATE CASCADE  
);
```

Função da Tabela

- Cria uma relação entre aluno e Turma
- Expressa histórico escolar e grade curricular do aluno

Mais de 1000 matrículas

- Alunos matriculados em turmas de disciplinas da Grade de seus cursos
- Sistema de **grade** é desenvolvido através de outra tabela

Tabela Leciona Criação da Tabela

```
CREATE TABLE leciona (  
  nusp_professor INTEGER NOT NULL,  
  sigla_disciplina VARCHAR(20) NOT NULL,  
  codigo_turma VARCHAR(5) NOT NULL,  
  vigencia VARCHAR(10) NOT NULL,  
  observacoes VARCHAR(40),  
  PRIMARY KEY (nusp_professor, sigla_disciplina, codigo_turma, vigencia)  
  FOREIGN KEY (nusp_professor) REFERENCES professor(nusp_associado)  
    ON DELETE SET NULL ON UPDATE CASCADE,  
  FOREIGN KEY (sigla_disciplina, codigo_turma, vigencia)  
    REFERENCES turma(sigla_disciplina, codigo_turma, vigencia)  
    ON DELETE SET NULL ON UPDATE CASCADE  
);
```

Função da Tabela

- Cria uma relação entre Professor e Turma

Mais de 60 turmas com docente

- Cada turma recebe um professor de um instituto, cujo departamento oferece a disciplina.
- Um monitor, não aluno da turma, também se relaciona com turma através de outra tabela **monitoria**

Principais Tabelas

Tabela Sala *Criação da Tabela*

```
CREATE TABLE sala (  
  id_sala VARCHAR(4) , --- (LETRANÚMERO)  
  lot_max INTEGER,  
  descricao VARCHAR(240),  
  id_instituto INTEGER,  
  PRIMARY KEY(id_sala, id_instituto),  
  FOREIGN KEY (id_instituto) REFERENCES instituto(id_instituto)  
    ON DELETE SET NULL ON UPDATE CASCADE  
);
```

Função da Tabela

- Representar uma sala onde serão ofertadas aulas de uma turma e possíveis reservas.

Mais de 280 salas

- Distribuídas entre os 7 institutos em 3 Blocos
- Lotação máxima distribuída de modo aleatório
- Recursos determinados através da **tabela de recursos**

Tabela Pedido_Reserva *Criação da Tabela*

```
CREATE TABLE pedido_reserva (  
  id_pedido SERIAL NOT NULL,  
  proposito TEXT NOT NULL,  
  horario_geracao TIMESTAMP NOT NULL,  
  data_reserva DATE NOT NULL,  
  hora_inicio_reserva TIME NOT NULL,  
  hora_fim_reserva TIME NOT NULL,  
  status SMALLINT CHECK(status in (1,2,3)), -- 1 Pendente, 2 Aprovado, 3 Reprovado  
  timestamp_veredito TIMESTAMP,  
  id_sala VARCHAR(4) NOT NULL,  
  id_instituto INTEGER NOT NULL,  
  nusp_solicitante INTEGER NOT NULL,  
  nusp_aprovador INTEGER,  
  PRIMARY KEY (id_pedido),  
  FOREIGN KEY (id_sala,id_instituto) REFERENCES sala(id_sala,id_instituto)  
    ON DELETE CASCADE ON UPDATE CASCADE,  
  FOREIGN KEY (nusp_solicitante) REFERENCES usuario(nusp)  
    ON DELETE CASCADE ON UPDATE CASCADE,  
  FOREIGN KEY (nusp_aprovador) REFERENCES funcionario(nusp_associado)  
    ON DELETE RESTRICT ON UPDATE CASCADE  
);
```

Função da Tabela

- Representação de um pedido para reserva de sala em um instituto.

Tabelas Auxiliares

Tabela Avaliação_Monitor

```
CREATE TABLE avaliacao_monitor (  
  nota DECIMAL CHECK (nota BETWEEN 0 AND 10) NOT NULL ,  
  comentario VARCHAR(240) NOT NULL,  
  nusp_aluno INTEGER NOT NULL,  
  nusp_monitor INTEGER NOT NULL,  
  time_stamp TIMESTAMP NOT NULL,  
  PRIMARY KEY (nusp_aluno, nusp_monitor, time_stamp),  
  FOREIGN KEY (nusp_aluno) REFERENCES aluno(nusp_usuario)  
    ON DELETE CASCADE ON UPDATE CASCADE,  
  FOREIGN KEY (nusp_monitor) REFERENCES monitor(nusp_aluno)  
    ON DELETE RESTRICT ON UPDATE CASCADE  
);
```

Tabela Pedido_Reserva

```
CREATE TABLE avaliacao_professor (  
  nota DECIMAL CHECK (nota BETWEEN 0 AND 10) NOT NULL,  
  comentario VARCHAR(240) NOT NULL,  
  nusp_aluno INTEGER NOT NULL,  
  nusp_professor INTEGER NOT NULL,  
  time_stamp TIMESTAMP NOT NULL,  
  PRIMARY KEY (nusp_aluno, nusp_professor, time_stamp),  
  FOREIGN KEY (nusp_aluno) REFERENCES aluno(nusp_usuario)  
    ON DELETE CASCADE ON UPDATE CASCADE,  
  FOREIGN KEY (nusp_professor) REFERENCES professor(nusp_associado)  
    ON DELETE CASCADE ON UPDATE CASCADE  
);
```

Tabela Coordenacao

```
CREATE TABLE coordenacao (  
  nusp_professor INTEGER,  
  id_departamento INTEGER,  
  data_inicio DATE,  
  data_fim DATE,  
  PRIMARY KEY(nusp_professor, id_departamento),  
  FOREIGN KEY (nusp_professor) REFERENCES professor(nusp_associado)  
    ON DELETE SET NULL ON UPDATE CASCADE,  
  FOREIGN KEY (id_departamento) REFERENCES departamento(id_departamento)  
    ON DELETE SET NULL ON UPDATE CASCADE  
);
```

Tabela Representa_Disciplina

```
CREATE TABLE representa_disciplina (  
  nusp_professor INTEGER NOT NULL ,  
  sigla_disciplina VARCHAR(20) NOT NULL ,  
  data_inicio DATE,  
  data_fim DATE,  
  PRIMARY KEY (sigla_disciplina, nusp_professor),  
  FOREIGN KEY (sigla_disciplina) REFERENCES disciplina(sigla)  
    ON DELETE CASCADE ON UPDATE CASCADE,  
  FOREIGN KEY (nusp_professor) REFERENCES professor(nusp_associado)  
    ON DELETE CASCADE ON UPDATE CASCADE  
);
```

Tabelas Auxiliares

Tabela Aluno

```
CREATE TABLE aluno (  
    num_matricula INTEGER UNIQUE NOT NULL,  
    nusp_usuario INTEGER PRIMARY KEY,  
    id_curso INTEGER,  
    FOREIGN KEY (nusp_usuario) REFERENCES usuario(nusp)  
        ON DELETE CASCADE ON UPDATE CASCADE,  
    FOREIGN KEY (id_curso) REFERENCES curso(id_curso)  
        ON DELETE SET NULL ON UPDATE CASCADE  
);
```

Tabela Monitoria

```
CREATE TABLE monitoria (  
    carga_horaria INTEGER CHECK (carga_horaria > 0) NOT NULL,  
    nusp_monitor INTEGER NOT NULL,  
    sigla_disciplina VARCHAR(20) NOT NULL,  
    codigo_turma VARCHAR(5) NOT NULL,  
    vigencia VARCHAR(10) NOT NULL,  
    PRIMARY KEY(nusp_monitor,sigla_disciplina,codigo_turma,vigencia),  
    FOREIGN KEY (nusp_monitor) REFERENCES monitor(nusp_aluno)  
        ON DELETE CASCADE ON UPDATE CASCADE,  
    FOREIGN KEY (sigla_disciplina, codigo_turma, vigencia)  
        REFERENCES turma(sigla_disciplina, codigo_turma, vigencia)  
        ON DELETE CASCADE ON UPDATE CASCADE  
);
```

Tabela Monitor

```
CREATE TABLE monitor (  
    nusp_aluno INTEGER PRIMARY KEY,  
    FOREIGN KEY (nusp_aluno) REFERENCES aluno(nusp_usuario)  
        ON DELETE CASCADE ON UPDATE CASCADE  
);
```

Tabela Associado

```
CREATE TABLE associado (  
    sala VARCHAR(15),  
    num INTEGER,  
    bloco VARCHAR(6),  
    nusp_usuario INTEGER PRIMARY KEY ,  
    FOREIGN KEY (nusp_usuario) REFERENCES usuario(nusp)  
        ON DELETE CASCADE ON UPDATE CASCADE  
);
```

Tabelas Auxiliares

Tabela Professor

```
CREATE TABLE professor (  
    nusp_associado INTEGER PRIMARY KEY,  
    FOREIGN KEY (nusp_associado) REFERENCES associado(nusp_usuario)  
        ON DELETE CASCADE ON UPDATE CASCADE  
);
```

Tabela Funcionário

```
CREATE TABLE funcionario (  
    nusp_associado INTEGER PRIMARY KEY,  
    FOREIGN KEY (nusp_associado) REFERENCES associado(nusp_usuario)  
        ON DELETE CASCADE ON UPDATE CASCADE  
);
```

Tabela Avaliação_Professor

```
CREATE TABLE avaliacao_professor (  
    nota DECIMAL CHECK (nota BETWEEN 0 AND 10) NOT NULL,  
    comentario VARCHAR(240) NOT NULL,  
    nusp_aluno INTEGER NOT NULL,  
    nusp_professor INTEGER NOT NULL,  
    time_stamp TIMESTAMP NOT NULL,  
    PRIMARY KEY (nusp_aluno, nusp_professor, time_stamp),  
    FOREIGN KEY (nusp_aluno) REFERENCES aluno(nusp_usuario)  
        ON DELETE CASCADE ON UPDATE CASCADE,  
    FOREIGN KEY (nusp_professor) REFERENCES professor(nusp_associado)  
        ON DELETE CASCADE ON UPDATE CASCADE  
);
```

Tabela Vínculo_Trabalho

```
CREATE TABLE vinculo_trabalho (  
    cargo VARCHAR(40) NOT NULL,  
    data_inicio DATE NOT NULL,  
    data_fim DATE,  
    tipo_vinculo SMALLINT CHECK (tipo_vinculo in (1,2)), -- 1 Temporário, 2 Efetivo  
    nusp_associado INTEGER NOT NULL,  
    id_instituto INTEGER,  
    PRIMARY KEY(nusp_associado, id_instituto),  
    FOREIGN KEY (nusp_associado) REFERENCES associado(nusp_usuario)  
        ON DELETE CASCADE ON UPDATE CASCADE,  
    FOREIGN KEY (id_instituto) REFERENCES instituto(id_instituto)  
        ON DELETE SET NULL ON UPDATE CASCADE  
);
```

Tabelas Auxiliares

Tabela Disciplina

```
CREATE TABLE disciplina (  
  sigla VARCHAR(20) PRIMARY KEY UNIQUE,  
  nome VARCHAR(150),  
  objetivo VARCHAR(250),  
  conteudo VARCHAR(250),  
  bibliografia VARCHAR(250),  
  id_departamento INTEGER,  
  FOREIGN KEY (id_departamento) REFERENCES departamento(id_departamento)  
    ON DELETE SET NULL ON UPDATE CASCADE  
);
```

Tabela Grade

```
CREATE TABLE grade (  
  tipo_disciplina SMALLINT NOT NULL CHECK (tipo_disciplina IN (1,2)),  
  id_curso INTEGER NOT NULL,  
  sigla_disciplina VARCHAR(20) NOT NULL,  
  PRIMARY KEY (id_curso, sigla_disciplina),  
  FOREIGN KEY (id_curso) REFERENCES curso(id_curso)  
    ON DELETE CASCADE ON UPDATE CASCADE,  
  FOREIGN KEY (sigla_disciplina) REFERENCES disciplina(sigla)  
    ON DELETE CASCADE ON UPDATE CASCADE  
);
```

Tabela Requisito

```
CREATE TABLE requisito (  
  sigla_disciplina_requerente VARCHAR(20) NOT NULL,  
  sigla_disciplina_requerida VARCHAR(20) NOT NULL,  
  PRIMARY KEY (sigla_disciplina_requerente, sigla_disciplina_requerida),  
  FOREIGN KEY (sigla_disciplina_requerente) REFERENCES disciplina(sigla)  
    ON DELETE CASCADE ON UPDATE CASCADE,  
  FOREIGN KEY (sigla_disciplina_requerida) REFERENCES disciplina(sigla)  
    ON DELETE CASCADE ON UPDATE CASCADE  
);
```

Tabelas Auxiliares

Tabela Oferta_Aula

```
CREATE TABLE oferta_aula (  
  id_oferta SERIAL PRIMARY KEY,  
  id_sala VARCHAR(4) NOT NULL,  
  id_instituto INTEGER NOT NULL,  
  sigla_disciplina VARCHAR(20) NOT NULL,  
  codigo_turma VARCHAR(5) NOT NULL,  
  vigencia VARCHAR(10) NOT NULL,  
  FOREIGN KEY (id_sala,id_instituto) REFERENCES sala(id_sala,id_instituto)  
    ON DELETE RESTRICT ON UPDATE CASCADE,  
  FOREIGN KEY (sigla_disciplina, codigo_turma, vigencia)  
    REFERENCES turma(sigla_disciplina, codigo_turma, vigencia)  
    ON DELETE RESTRICT ON UPDATE CASCADE  
);
```

Tabela Horário_Aula

```
CREATE TABLE horario_aula (  
  id_oferta INTEGER PRIMARY KEY,  
  dia_semana TEXT,  
  hora_inicio TIME,  
  hora_fim TIME,  
  FOREIGN KEY (id_oferta) REFERENCES oferta_aula(id_oferta)  
    ON DELETE CASCADE ON UPDATE CASCADE  
);
```

Tabela Monitoria

```
CREATE TABLE monitoria (  
  carga_horaria INTEGER CHECK (carga_horaria > 0) NOT NULL,  
  nusp_monitor INTEGER NOT NULL,  
  sigla_disciplina VARCHAR(20) NOT NULL,  
  codigo_turma VARCHAR(5) NOT NULL,  
  vigencia VARCHAR(10) NOT NULL,  
  PRIMARY KEY(nusp_monitor,sigla_disciplina,codigo_turma,vigencia),  
  FOREIGN KEY (nusp_monitor) REFERENCES monitor(nusp_aluno)  
    ON DELETE CASCADE ON UPDATE CASCADE,  
  FOREIGN KEY (sigla_disciplina, codigo_turma, vigencia)  
    REFERENCES turma(sigla_disciplina, codigo_turma, vigencia)  
    ON DELETE CASCADE ON UPDATE CASCADE  
);
```

Tabela Coordenação

```
CREATE TABLE coordenacao (  
  nusp_professor INTEGER,  
  id_departamento INTEGER,  
  data_inicio DATE,  
  data_fim DATE,  
  PRIMARY KEY(nusp_professor,id_departamento),  
  FOREIGN KEY (nusp_professor) REFERENCES professor(nusp_associado)  
    ON DELETE SET NULL ON UPDATE CASCADE,  
  FOREIGN KEY (id_departamento) REFERENCES departamento(id_departamento)  
    ON DELETE SET NULL ON UPDATE CASCADE  
);
```

Povoamento

Tabela Usuário

- Povoado através de softwares de geração de dados.
- Mantido a **unicidade** em e-mails, nusps e RG.

Tabela Aluno e Monitor

- E-mails com padrão LIKE
'nome_sobrenome@usp.br'
- NUSP com **8 dígitos**
- Distribuição **aleatória uniforme** nos cursos
- Matrícula **aleatória (6 dígitos)**

num_matricula integer	nusp_usuario [PK] integer	id_curso integer
638697	49018460	3
901133	99617579	10
418982	27027311	10
933858	64922718	2
381056	21971760	7

Inserção Aleatória de Monitores

```
--- Comando para inserção aleatória de 65 monitores
INSERT INTO monitor (nusp_aluno)
SELECT nusp_usuario FROM aluno
    WHERE nusp_usuario BETWEEN 10000000 AND 99999999
    ORDER BY RANDOM() LIMIT 65
ON CONFLICT (nusp_aluno) DO NOTHING;
```

- Esse tipo de inserção permite que monitores possam ser de diversos outros institutos e cursos.
- **Regras ativas** posteriores garantem que tais monitores não sejam alunos que estejam matriculados em tal turma que irão monitorar.

Povoamento

Tabela Funcionário

- E-mails com padrão LIKE `'func_nome@usp.br'`
- NUSP pequenos
- Distribuição **aleatória uniforme** nos institutos em diversos cargos.
- Salas aleatórias distribuídas entre os blocos do instituto

nome character varying (100) 🔒	email character varying (150) 🔒	nusp integer 🔒	cargo character varying (40) 🔒	data_inicio date 🔒	data_fim date 🔒
Nathalia Reis	funcionario_nathaliareis@usp.br	1	Assistente Administrativo	2021-03-15	[null]
Gustavo Costa	funcionario_gustavocosta@usp.br	5	Auxiliar de Serviços Ger...	2021-09-25	[null]
Fernanda Tavares	funcionario_fernandatavares@usp.br	6	Motorista	2021-10-10	[null]
Maria Eduarda Nunes	funcionario_mariaeduardanunes@usp...	7	Recepcionista	2021-11-15	[null]
Renata Batista	funcionario_renatabatista@usp.br	8	Analista de Dados	2021-12-01	[null]
Roberta Azevedo	funcionario_robertaazevedo@usp.br	10	Analista de TI	2021-02-14	[null]
Susana Dias	funcionario_susanadias@usp.br	11	Assistente Administrativo	2021-04-01	[null]
Tácio Neto	funcionario_tacioneto@usp.br	12	Secretário Executivo	2021-06-05	[null]
Ulisses Fogaça	funcionario_ulissesfogaça@usp.br	13	Técnico de Laboratório	2021-07-20	[null]
Verônica Duarte	funcionario_veronicaduarte@usp.br	14	Bibliotecário	2021-09-01	[null]

Povoamento

Tabela Professor

- E-mails com padrão **LIKE** *'prof_nome@usp.br'*
- NUSP com 5 dígitos
- Distribuição **aleatória uniforme** nos institutos em diversos cargos.
- Salas aleatórias distribuídas entre os blocos do instituto

nome character varying (100) 🔒	email character varying (150) 🔒	nusp integer 🔒	cargo character varying (40) 🔒	data_inicio date 🔒	data_fim date 🔒
Alice Monteiro	prof_monteiro@usp.br	43276	Professor Nível 1	2018-01-10	[null]
Paula Neves	prof_neves@usp.br	20862	Professor Nível 1	2018-03-10	[null]
Bernardo Albuquerque	prof_albuquerque@usp.br	33527	Professor Nível 1	2018-05-10	[null]
Camila Cardoso	prof_cardoso@usp.br	29064	Professor Nível 1	2018-07-10	[null]
Diego Figueiredo	prof_figueiredo@usp.br	39924	Professor Nível 1	2018-09-10	[null]
Eduarda Campos	prof_campos@usp.br	32393	Professor Nível 2	2019-01-10	[null]
Felipe Tavares	prof_tavares@usp.br	81551	Professor Nível 2	2019-03-10	[null]
Gabriela Martins	prof_martins@usp.br	40794	Professor Nível 2	2019-05-10	[null]
Henrique Rocha	prof_rocha@usp.br	22686	Professor Nível 2	2019-07-10	[null]
Isabela Dias	prof_dias@usp.br	37166	Professor Nível 2	2019-09-10	[null]
João Pereira	prof_pereira@usp.br	48938	Professor Nível 3	2020-01-10	[null]

Povoamento

Tabela Instituto

- Povoado através de dados existentes dos institutos **vizinhos do IME**.

Tabela Curso e Departamento

- Definidas através da **simplificação de dados** já existentes de cursos e departamentos para cada instituto alocado no BD.

Tabelas de Avaliação

- *Avaliação de Monitor, Sala e Professores*
- *Comentários gerados através de ferramentas de IA*
- *Notas distribuídas segundo uma distribuição de probabilidade no R*
- *Tempo, Avaliadores e Avaliados aleatórios mediante aos vínculos (**Regras Ativas**)*

Tabela Disciplina, Grade e Requisito

- Dados obtidos através do sistema Júpiter Web próprio da USP.

Tabela Turma e Participa_em

- **Turmas** definidas através da disposição das disciplinas em seus semestres de vigência datadas pela USP, podendo haver repetição por mudança de sigla
- **Matrícula** dos alunos definidas através da disposição dos alunos em disciplinas de sua grade do conclusão.
- Conforme o estado do BD é demonstrado no **segundo semestre**, os dados do primeiro semestre são *completos* e definem a inclusão dos demais no segundo.

Tabela Sala e Recursos

- Suposições de 3 blocos por instituto (A,B,C)
- Disposição de salas nos 3 blocos de modo (60,20,10)
- Lotação máxima e recursos foram dispostos de forma aleatória

Tabelas de Reserva

- Criada e mantida sob regras ativas de uso
- Comentários gerados por assistentes de IA
- Status de Pendente, Ativo ou Negado, definido de modo aleatório.

Tabela Horário_Aula e Oferta_Aula

- Salas aleatórias do instituto que possui a turma
- Horários de aula distribuídos mediante os cursos em 2 dias da semana.

Tabelas de Coordenação e Leciona

- **Coordena:** Distribuição aleatória dos professores de nível 1 ou 2 em departamentos pertencentes ao instituto do seu vínculo de trabalho
- **Leciona:** Distribuição aleatório dos professores do institutos sobre as turmas lecionadas no mesmo

nome character varying (100) 🔒	nusp integer 🔒	sigla character varying (20) 🔒
Paula Neves	20862	EST
Henrique Rocha	22686	MAP
Camila Cardoso	29064	MAC
Eduarda Campos	32393	MAT
Bernardo Albuquerque	33527	DCC
Vinícius Melo	26427	ECO
Yasmin Cunha	42436	DAC
Lucas Marques	29276	DAP

Professores de um Instituto

Exemplo para o IME

nome character varying (100)	email character varying (150)	nusp [PK] integer
Alice Monteiro	prof_monteiro@usp.br	43276
Bernardo Albuquerque	prof_albuquerque@usp...	33527
Camila Cardoso	prof_cardoso@usp.br	29064
Diego Figueiredo	prof_figueiredo@usp.br	39924
Eduarda Campos	prof_campos@usp.br	32393
Felipe Tavares	prof_tavares@usp.br	81551
Gabriela Martins	prof_martins@usp.br	40794
Henrique Rocha	prof_rocha@usp.br	22686
Isabela Dias	prof_dias@usp.br	37166
João Pereira	prof_pereira@usp.br	48938
Larissa Souza	prof_souza@usp.br	38996
Marcelo Lima	prof_lima@usp.br	66999
Natália Costa	prof_costa@usp.br	43624
Otávio Oliveira	prof_oliveira@usp.br	20040
Paula Neves	prof_neves@usp.br	20862

Código em PGSQL

```
--- Selecionar professores dentro de um instituto na USP :
SELECT u.nome, u.email, u.nusp FROM vinculo_trabalho as v
      JOIN usuario as u ON u.nusp = v.nusp_associado
      WHERE v.id_instituto = 1
      AND v.cargo LIKE 'Professor %'      -- Id do instituto IME
      ORDER BY u.nome, v.cargo ;
```

Turmas com maiores rendimentos

Para o 2º Semestre

sigla_disciplina character varying (20) 🔒	codigo_turma character varying (5) 🔒	media_final numeric 🔒
CRP0349	T02	8.2200000000000000
4310111	T01	7.3433333333333333
MAT0112	T01	6.8875000000000000
MAT1351	T01	6.8085714285714286
DFD0152	T02	6.6700000000000000
EAD0610	T01	6.5575000000000000
MAT2453	T01	6.4960000000000000
4300151	T01	6.4569230769230769
4302111	T01	6.3366666666666667
MAE0111	T01	6.1666666666666667

Código em PGSQL

```
--- Selecionar a sala com maiores notas:
SELECT medias.sigla_disciplina,
       medias.codigo_turma,
       medias.media_final FROM
  (SELECT AVG(p.media_final) AS media_final,
    p.sigla_disciplina,
    p.codigo_turma
  FROM participa_em AS p
 WHERE p.vigencia = '2025-1'
  GROUP BY
    p.sigla_disciplina,
    p.codigo_turma,
    p.vigencia
  ) AS medias
ORDER BY medias.media_final DESC LIMIT 10;
```

Consultas

Pedidos de Reserva pendentes

id_pedido [PK] integer	proposito text	nusp_solicitante integer
1	Estudo intensivo para a prova de Cálculo 3, rezando para que a derivada ajude.	51965656
2	Reunião do time de desenvolvimento de software para tentar convencer o código a funci...	11704071
3	Prática de apresentação para o seminário de TCC. Preciso de uma sala com espelho.	42318353
4	Sessão de brainstorming para um projeto de inovação (e encomendar pizza em paz).	58860446

Código em PGSQL

```
--- Selecionar os pedidos pendentes de sala:  
SELECT id_pedido, proposito, nusp_solicitante  
FROM pedido_reserva  
WHERE status = 1; --- Pendente
```

Buscar uma sala com tais recursos

- Salas que possuem 3 ar-condicionados dentro do IME

id_sala [PK] character varying (4)	id_instituto [PK] integer
A3	1
A7	1
B3	1
B9	1
B15	1
B21	1
B27	1
B33	1
C2	1

Código em PGSQL

```
--- Selecionar as salas que possuem determinado recurso
SELECT s.id_sala, s.id_instituto FROM sala AS s
      JOIN recursos AS r ON s.id_sala = r.id_sala
      AND s.id_instituto = r.id_instituto
      WHERE r.n_ar_condicionado = 3
      AND s.id_instituto = 1 ; --- IME
```

Consultas

Localizar sala e horários de uma turma

- Exemplo para disciplina 2100102 (Sistema Oceano II)

id_sala character varying (4) 🔒	id_instituto integer 🔒	nome character varying (150) 🔒	sigla_disciplina character varying (20) 🔒	dia_semana text 🔒	hora_inicio time without time zone 🔒	hora_fim time without time zone 🔒
A5	6	Sistema Oceano II	2100102	Quarta-feira	16:00:00	17:40:00
A5	6	Sistema Oceano II	2100102	Segunda-feira	16:00:00	17:40:00

Código em PGSQL

```
--- Selecionar o horário e dias em que uma disciplina é oferecida (Sistema Oceano II)
SELECT DISTINCT s.id_sala, o.id_instituto, d.nome, o.sigla_disciplina, h.dia_semana,
h.hora_inicio,h.hora_fim FROM
oferta_aula AS o JOIN horario_aula AS h ON h.id_oferta = o.id_oferta
JOIN disciplina as d ON o.sigla_disciplina = d.sigla
JOIN sala as s ON o.id_sala = s.id_sala
WHERE o.sigla_disciplina = '2100102';
```

Operações Frequentes

Mudar média final e frequência de um aluno em determinada turma

```
UPDATE participa_em
SET frequencia = 75
WHERE nusp_aluno = 10314744
    AND sigla_disciplina = 'AUH2502'
    AND codigo_turma = 'T01'
    AND vigencia = '2025-2';
```

Atualizar Recursos de uma Sala

```
UPDATE recursos
SET n_ar_condicionado = 3
WHERE id_sala = 'A2' AND id_instituto = 1;
```

Mudar o estado de um pedido

```
UPDATE pedido_reserva
SET status = 2
WHERE id_pedido = 1;
```

Atualizar cargo de um professor

```
UPDATE vinculo_trabalho
SET cargo = 'Professor Nível 2'
WHERE nusp_associado = 43276 AND data_fim IS NULL;
```

```
UPDATE vinculo_trabalho
SET data_inicio = CURRENT_DATE
WHERE nusp_associado = 43276 AND data_fim IS NULL;
```

Operações Frequentes

Tornar matrículas desativadas

```
UPDATE participa_em  
SET status = 2  
WHERE nusp_aluno = 10314744  
AND sigla_disciplina = 'AUH2502'
```

Excluir matrículas desativadas

```
DELETE FROM participa_em  
WHERE status = 2;
```

Remoção de Histórico

```
DELETE FROM monitoria  
WHERE vigencia < '2024-1';
```

Correção em massa

```
UPDATE participa_em  
SET media_final = media_final + 0.5  
WHERE sigla_disciplina = 'MAE0121'  
AND codigo_turma = 'T01'  
AND vigencia = '2025-1'  
AND media_final <= 10.0;
```

Inserção de novo usuário

```
-- Inserção na tabela geral de usuários  
INSERT INTO usuario (email, senha, nusp, rg, foto, nome, pnome, unome) VALUES  
('kadubarille@usp.br', '1234BASQUETE', 20250001, 123467221, NULL, 'Kadu Barrille', 'Kadu', 'Barrille');
```

Desempenho geral do Semestre

Demonstra a média geral, a disciplina e o código da turma para todas as turmas ministradas em um semestre.

```
SELECT * FROM Desempenho_Geral ORDER BY RANDOM() LIMIT 8;
```

sigla_disciplina character varying (20) 🔒	codigo_turma character varying (5) 🔒	media numeric 🔒
MAT0146	T01	6.0416666666666667
IOB0167	T01	5.3469230769230769
MAC0121	T01	5.6450000000000000
4310111	T01	5.9833333333333333
EAC0111	T01	7.2233333333333333
DFD0152	T02	5.1700000000000000
AUP0608	T01	7.0671428571428571
MAT0112	T01	6.0087500000000000

Código em PGSQL

```
CREATE VIEW Desempenho_Geral AS
SELECT sigla_disciplina, codigo_turma,
       AVG(media_final) AS media
FROM participa_em
WHERE vigencia LIKE '%1'
GROUP BY sigla_disciplina, codigo_turma ;
```

Histórico Geral dos Alunos

```
SELECT * FROM Historico_Aluno ORDER BY RANDOM() LIMIT 10;
```

Código em PGSQL

```
CREATE VIEW Historico_Aluno AS SELECT U.nome, U.nusp,  
    AVG(media_final) AS media_geral,  
    AVG(frequencia) AS frequencia  
FROM usuario U JOIN participa_em p  
ON p.nusp_aluno = U.nusp  
GROUP BY U.nusp;
```

nome character varying (100) 🔒	nusp integer 🔒	media_geral numeric 🔒	frequencia numeric 🔒
Larissa Moreira	78394438	5.0766666666666667	85.1000000000000000
Camila Moreira	63703414	7.0766666666666667	85.2000000000000000
Isabela Andrade	92917882	4.6366666666666667	83.3333333333333333
Matheus Souza	84394747	7.5233333333333333	77.0666666666666667
Felipe Lima	31213864	4.4040000000000000	81.4200000000000000
Fernando Peixoto	79877819	5.0300000000000000	81.6333333333333333
Fernanda Santos	27743327	6.1050000000000000	84.7750000000000000
Sandra Rocha	24882813	5.4120000000000000	83.9600000000000000
Ana Castro	24342552	5.7166666666666667	81.2333333333333333
Rafael Gomes	84220836	4.9333333333333333	76.2666666666666667

Visões

Grade Horária de um Aluno

sigla_disciplina character varying (20) 🔒	codigo_turma character varying (5) 🔒	dia_semana text 🔒	hora_inicio time without time zone 🔒	hora_fim time without time zone 🔒	id_sala character varying (4) 🔒
MAE0212	T01	Quarta-feira	08:00:00	09:40:00	B17
MAE0212	T01	Segunda-feira	08:00:00	09:40:00	B17
MAE0225	T01	Quarta-feira	14:00:00	15:40:00	B26
MAE0225	T01	Segunda-feira	14:00:00	15:40:00	B26
MAT3211	T01	Quarta-feira	14:00:00	15:40:00	B15
MAT3211	T01	Segunda-feira	14:00:00	15:40:00	B15

Código em PGSQL

```
CREATE OR REPLACE VIEW Grade_Aluno AS
SELECT DISTINCT p.sigla_disciplina,
p.codigo_turma, h.dia_semana, h.hora_inicio, h.hora_fim, o.id_sala
FROM participa_em AS p JOIN oferta_aula AS o
ON p.codigo_turma = o.codigo_turma AND p.sigla_disciplina = o.sigla_disciplina
JOIN horario_aula AS h ON h.id_oferta = o.id_oferta
WHERE nusp_aluno = 84175446
ORDER BY p.sigla_disciplina
```

Visões

Buscar salas por adequação

Expressa salas que comportam n pessoas e possuem tal número de recursos

id_sala character varying (4) 🔒	sigla character varying (10) 🔒	lot_max integer 🔒	n_ar_condicionado smallint 🔒	n_lousas smallint 🔒
A5	IO	60	3	2
A5	IF	60	3	2
A5	FEA	60	3	2
B17	IME	59	3	2
B17	FAU	58	3	2
B17	FEA	57	3	2

Código em PGSQL

```
SELECT s.id_sala, i.sigla, s.lot_max, r.n_ar_condicionado, r.n_lousas
FROM sala AS s JOIN recursos AS r
ON r.id_sala = s.id_sala
JOIN instituto AS i
ON s.id_instituto = i.id_instituto
WHERE
lot_max > 50 AND
r.n_ar_condicionado > 2 AND
r.n_lousas > 1
ORDER BY lot_max DESC;
```

Visões

Monitorias de 2025-2

```
SELECT * FROM View_Monitorias_Por_Disciplina_2025_2 ORDER BY RANDOM() LIMIT 6;
```

nome_monitor character varying (100) 🔒	nome_disciplina character varying (150) 🔒	codigo_turma character varying (5) 🔒	carga_horaria integer 🔒
Rafael Ribeiro	Cálculo Diferencial e Integral I	T01	80
Patrícia Santos	Princípios de Desenvolvimento de Algorit...	T02	80
Rodrigo Peixoto	Teoria das Organizações	T02	80
Amanda Souza	Álgebra Linear I	T02	80
Ana Castro	Técnicas de Programação I	T01	80
Aline Machado	Princípios de Desenvolvimento de Algorit...	T01	80

Código em PGSQL

```
CREATE OR REPLACE VIEW View_Monitorias_Por_Disciplina_2025_2 AS
SELECT
    u.nome AS nome_monitor,
    d.nome AS nome_disciplina,
    m.codigo_turma,
    m.carga_horaria
FROM monitoria m JOIN usuario u ON m.nusp_monitor = u.nusp
JOIN disciplina d ON m.sigla_disciplina = d.sigla
WHERE m.vigencia = '2025-2'
ORDER BY |d.nome, u.nome;
```

Stored Procedures

Atualizar registros de matrícula de alunos

Exemplo com mudança da média final em uma turma (Operação Frequente)

Parte 1.

```
--- 1.Atualizar a media_final de um aluno
CREATE OR REPLACE FUNCTION atualizar_nota_aluno(
    p_nusp_aluno INT,
    p_sigla_disc VARCHAR,
    p_cod_turma INT,
    p_vigencia VARCHAR,
    p_media_final NUMERIC
) RETURNS VOID AS $$
```

Parte 2.

```
BEGIN
    UPDATE participa_em
    SET media_final = p_media_final
    WHERE nusp_aluno = p_nusp_aluno
        AND sigla_disciplina = p_sigla_disc
        AND codigo_turma = p_cod_turma
        AND vigencia = p_vigencia;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Registro de matrícula não encontrado.';
    END IF;
END;
$$ LANGUAGE plpgsql;
```

Stored Procedures

Relatório detalhado de turma

Retorna um feedback detalhado das notas e frequência dos alunos em uma turma

Parte 1.

```
CREATE OR REPLACE FUNCTION gerar_relatorio_turma(  
  p_sigla_disc VARCHAR,  
  p_cod_turma VARCHAR,  
  p_vigencia VARCHAR  
)  
RETURNS TABLE (  
  nusp_aluno INT,  
  nome_aluno VARCHAR,  
  media_final NUMERIC,  
  frequencia NUMERIC,  
  status VARCHAR  
) AS $$
```

Parte 2.

```
BEGIN  
  RETURN QUERY  
  SELECT  
    p.nusp_aluno,  
    u.nome,  
    p.frequencia,  
    p.media_final,  
    (CASE  
      WHEN p.media_final IS NULL THEN 'Pendente'  
      WHEN p.media_final >= 5.0 AND p.frequencia >70 THEN 'Aprovado'  
      ELSE 'Reprovado'  
    END)::VARCHAR(50) AS status  
  FROM  
    participa_em AS p  
  JOIN  
    usuario u ON p.nusp_aluno = u.nusp  
  WHERE  
    p.sigla_disciplina = p_sigla_disc  
    AND p.codigo_turma = p_cod_turma  
    AND p.vigencia = p_vigencia  
  ORDER BY  
    u.nome;  
END;  
$$ LANGUAGE plpgsql;
```

nusp_aluno integer	nome_aluno character varying	media_final numeric	frequencia numeric	status character varying
57040254	Amanda Souza	79.70	2.59	Reprovado
33043446	André Araujo	94.70	4.27	Reprovado
20056966	André Castro	79.80	7.20	Aprovado
61106721	André Queiroz	77.10	4.72	Reprovado

Stored Procedures

Transferência de funcionário para outro Instituto

Parte 1.

```
-- 7. Transfere um funcionário para outro instituto
CREATE OR REPLACE FUNCTION transferir_funcionario(
    p_nusp INT,
    p_novo_instituto_id INT
) RETURNS VOID AS $$
BEGIN
    -- Encerrar o vínculo de trabalho atual
    UPDATE vinculo_trabalho
    SET data_fim = CURRENT_DATE
    WHERE nusp_associado = p_nusp
    AND data_fim IS NULL;
```

Parte 2.

```
-- Inserir o novo vínculo no novo instituto, mantendo o cargo
INSERT INTO vinculo_trabalho (nusp_associado, id_instituto, cargo, data_inicio)
SELECT
    p_nusp,
    p_novo_instituto_id, -- Novo Instituto
    vt.cargo,            -- Mantém o cargo
    CURRENT_DATE
FROM
    vinculo_trabalho vt
WHERE
    nusp_associado = p_nusp
ORDER BY
    data_fim DESC
LIMIT 1;

IF NOT FOUND THEN
    RAISE EXCEPTION 'Funcionário não possui vínculo de trabalho anterior para transferência.';
END IF;
END;
$$ LANGUAGE plpgsql;
```

Stored Procedures

Agenda do Instituto

Retorna a grade de aulas em um dia da semana para o instituto em um semestre

Parte 1.

```
CREATE OR REPLACE FUNCTION aulas_por_dia_semana(  
    p_dia_semana VARCHAR,  
    p_vigencia VARCHAR,  
    p_id_instituto INTEGER  
)  
RETURNS TABLE (  
    disciplina VARCHAR,  
    codigo_turma VARCHAR,  
    id_sala VARCHAR,  
    id_instituto INTEGER,  
    hora_inicio TIME,  
    hora_fim TIME  
) AS $$
```

Parte 2.

```
BEGIN  
    RETURN QUERY  
    SELECT  
        d.nome,  
        oa.codigo_turma,  
        oa.id_sala,  
        i.id_instituto,  
        ha.hora_inicio,  
        ha.hora_fim  
    FROM  
        horario_aula ha  
    JOIN  
        oferta_aula oa ON ha.id_oferta = oa.id_oferta  
    JOIN  
        disciplina d ON oa.sigla_disciplina = d.sigla  
    JOIN  
        departamento dep ON d.id_departamento = dep.id_departament  
    JOIN  
        instituto i ON dep.id_instituto = i.id_instituto  
    WHERE  
        ha.dia_semana ILIKE p_dia_semana  
        AND oa.vigencia = p_vigencia  
        AND i.id_instituto = p_id_instituto  
    ORDER BY  
        ha.hora_inicio, d.nome;  
END;  
$$ LANGUAGE plpgsql;
```

Stored Procedures

Atribuir aluno como monitor em uma turma

Parte 1.

```
-- 5. Atribuir um aluno como monitor a uma turma
CREATE OR REPLACE FUNCTION atribuir_monitor(
    p_nusp_monitor INT,
    p_sigla_disc VARCHAR,
    p_cod_turma INT,
    p_vigencia VARCHAR,
    p_carga_horaria INT
) RETURNS VOID AS $$
```

Parte 2.

```
BEGIN
    INSERT INTO monitoria (carga_horaria, nusp_monitor, sigla_disciplina, codigo_turma, vigencia)
    VALUES (p_carga_horaria, p_nusp_monitor, p_sigla_disc, p_cod_turma, p_vigencia);
END;
$$ LANGUAGE plpgsql;
```

Triggers

Garantia de datas certas no Vínculo de Trabalho

Parte 1.

```
-- 1. Trigger para garantir que data_fim > data_inicio em Vínculo de Trabalho
CREATE OR REPLACE FUNCTION trg_check_data_vinculo()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.data_fim IS NOT NULL AND NEW.data_fim < NEW.data_inicio THEN
        RAISE EXCEPTION 'A data de fim do vínculo deve ser posterior à data de início.';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

Parte 2.

```
CREATE TRIGGER trg_vinculo_data_check
BEFORE INSERT OR UPDATE ON vinculo_trabalho FOR EACH ROW
EXECUTE FUNCTION trg_check_data_vinculo();
```

Triggers

Garantia de monitoria por monitor válido

Esse trigger garante que o monitor escolhido não esteja matriculado em tal turma

Parte 1.

```
CREATE OR REPLACE FUNCTION trg_check_monitor_matricula_conflito()  
RETURNS TRIGGER AS $$  
BEGIN  
    IF EXISTS (  
        SELECT 1  
        FROM participa_em AS p  
        WHERE p.nusp_aluno = NEW.nusp_monitor  
              AND p.sigla_disciplina = NEW.sigla_disciplina  
              AND p.codigo_turma = NEW.codigo_turma  
              AND p.vigencia = NEW.vigencia  
    ) THEN  
        RAISE EXCEPTION 'O aluno monitor não pode estar matriculado na mesma turma que irá monitorar.';  
    END IF;  
    RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;
```

Parte 2.

```
CREATE TRIGGER trg_monitor_matricula_conflict  
BEFORE INSERT ON monitoria FOR EACH ROW  
    EXECUTE FUNCTION trg_check_monitor_matricula_conflito();
```

Triggers

Check de Pré-Requisitos para matrícula

Essa regra ativa garante que as disciplinas requisitos sejam finalizadas e aprovadas para matrícula posterior

Parte 1.

```
-- 6. Checa Pré-Requisitos antes da matrícula (usando fn_checar_pre_requisito_aprovado)
CREATE OR REPLACE FUNCTION trg_check_prerequisitos_matricula()
RETURNS TRIGGER AS $$
DECLARE
    v_pre_req_sigla VARCHAR;
BEGIN
    FOR v_pre_req_sigla IN
        SELECT sigla_disciplina_requerida
        FROM requisito
        WHERE sigla_disciplina_requerente = NEW.sigla_disciplina
    LOOP
        IF NOT fn_checar_pre_requisito_aprovado(NEW.nusp_aluno, v_pre_req_sigla) THEN
            RAISE EXCEPTION 'Matrícula negada:
0 aluno não foi aprovado no pré-requisito: %.', v_pre_req_sigla;
        END IF;
    END LOOP;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

Parte 2.

```
CREATE TRIGGER trg_check_prerequisitos
BEFORE INSERT ON participa_em FOR EACH ROW
EXECUTE FUNCTION trg_check_prerequisitos_matricula();
```

Triggers

Curso único

Essa regra ativa define o critério do Banco de Dados de registro de um único curso para um aluno

Parte 1.

```
-- 9. Garante que um aluno só possa ter um curso (assumindo que a matrícula é na tabela 'aluno')
CREATE OR REPLACE FUNCTION trg_enforce_single_course()
RETURNS TRIGGER AS $$
BEGIN
    IF EXISTS (
        SELECT 1
        FROM aluno
        WHERE nusp_aluno = NEW.nusp_aluno
        AND id_curso IS NOT NULL
    ) THEN
        RAISE EXCEPTION 'O aluno NUSP % já está matriculado em um curso.', NEW.nusp_aluno;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

Parte 2.

```
CREATE TRIGGER trg_single_course_enrollment
BEFORE INSERT ON aluno FOR EACH ROW WHEN (NEW.id_curso IS NOT NULL)
    EXECUTE FUNCTION trg_enforce_single_course();
```

Triggers

Triggers de formatação

Sigla Disciplina em Maiúscula

```
-- 2. Trigger para formatar a sigla da disciplina em MAIÚSCULA
CREATE OR REPLACE FUNCTION trg_format_disciplina_sigla()
RETURNS TRIGGER AS $$
BEGIN
    NEW.sigla := UPPER(NEW.sigla);
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_upper_sigla
BEFORE INSERT OR UPDATE OF sigla ON disciplina FOR EACH ROW
EXECUTE FUNCTION trg_format_disciplina_sigla();
```

Nome Inicial com maiúscula

```
-- 7. Padroniza o nome do usuário (Inicial Maiúscula)
CREATE OR REPLACE FUNCTION trg_format_user_name()
RETURNS TRIGGER AS $$
BEGIN
    NEW.nome := INITCAP(NEW.nome);
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_format_name
BEFORE INSERT OR UPDATE OF nome ON usuario
FOR EACH ROW
EXECUTE FUNCTION trg_format_user_name();
```

Triggers

Trigger de Avaliação (1)

Essa regra ativa define um critério para que o aluno possa avaliar o professor de sua turma

Parte 1.

```
CREATE OR REPLACE FUNCTION trg_check_aluno_matriculado_avaliacao()  
RETURNS TRIGGER AS $$  
BEGIN  
    IF NOT EXISTS (  
        SELECT 1  
        FROM participa_em AS p  
        JOIN leciona l ON p.sigla_disciplina = l.sigla_disciplina  
            AND p.codigo_turma = l.codigo_turma  
            AND p.vigencia = l.vigencia  
        WHERE p.nusp_aluno = NEW.nusp_aluno  
            AND l.nusp_professor = NEW.nusp_professor  
            AND l.vigencia = NEW.vigencia  
    ) THEN  
        RAISE EXCEPTION 'O aluno não está elegível para avaliar este professor nesta vigência.';  
    END IF;  
    RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;
```

Parte 2.

```
CREATE TRIGGER trg_check_avaliacao_eligibility  
BEFORE INSERT ON avaliacao_professor FOR EACH ROW  
EXECUTE FUNCTION trg_check_aluno_matriculado_avaliacao();
```

Triggers

Trigger de Avaliação (2)

Essa regra ativa define um critério para que o aluno possa avaliar o monitor de sua turma

Parte 1.

```
CREATE OR REPLACE FUNCTION trg_check_aluno_matriculado_avaliacao_monitor()  
RETURNS TRIGGER AS $$  
BEGIN  
    IF NOT EXISTS (  
        SELECT 1  
        FROM participa_em AS p  
        JOIN monitoria m ON p.sigla_disciplina = m.sigla_disciplina  
            AND p.codigo_turma = m.codigo_turma  
            AND p.vigencia = m.vigencia  
        WHERE p.nusp_aluno = NEW.nusp_aluno  
            AND m.nusp_aluno = NEW.nusp_monitor  
            AND m.vigencia = NEW.vigencia  
    ) THEN  
        RAISE EXCEPTION 'O aluno não está elegível para avaliar este monitor nesta vigência.';  
    END IF;  
    RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;
```

Parte 2.

```
CREATE TRIGGER trg_check_avaliacao_eligibility_monitor  
BEFORE INSERT ON avaliacao_monitor FOR EACH ROW  
EXECUTE FUNCTION trg_check_aluno_matriculado_avaliacao_monitor();
```

Análise Estatística dos Dados

Análise de Dados de matrículas

Conexão no RSTUDIO

```
library("DBI")
library("RPostgres")
drv <- dbDriver("Postgres")

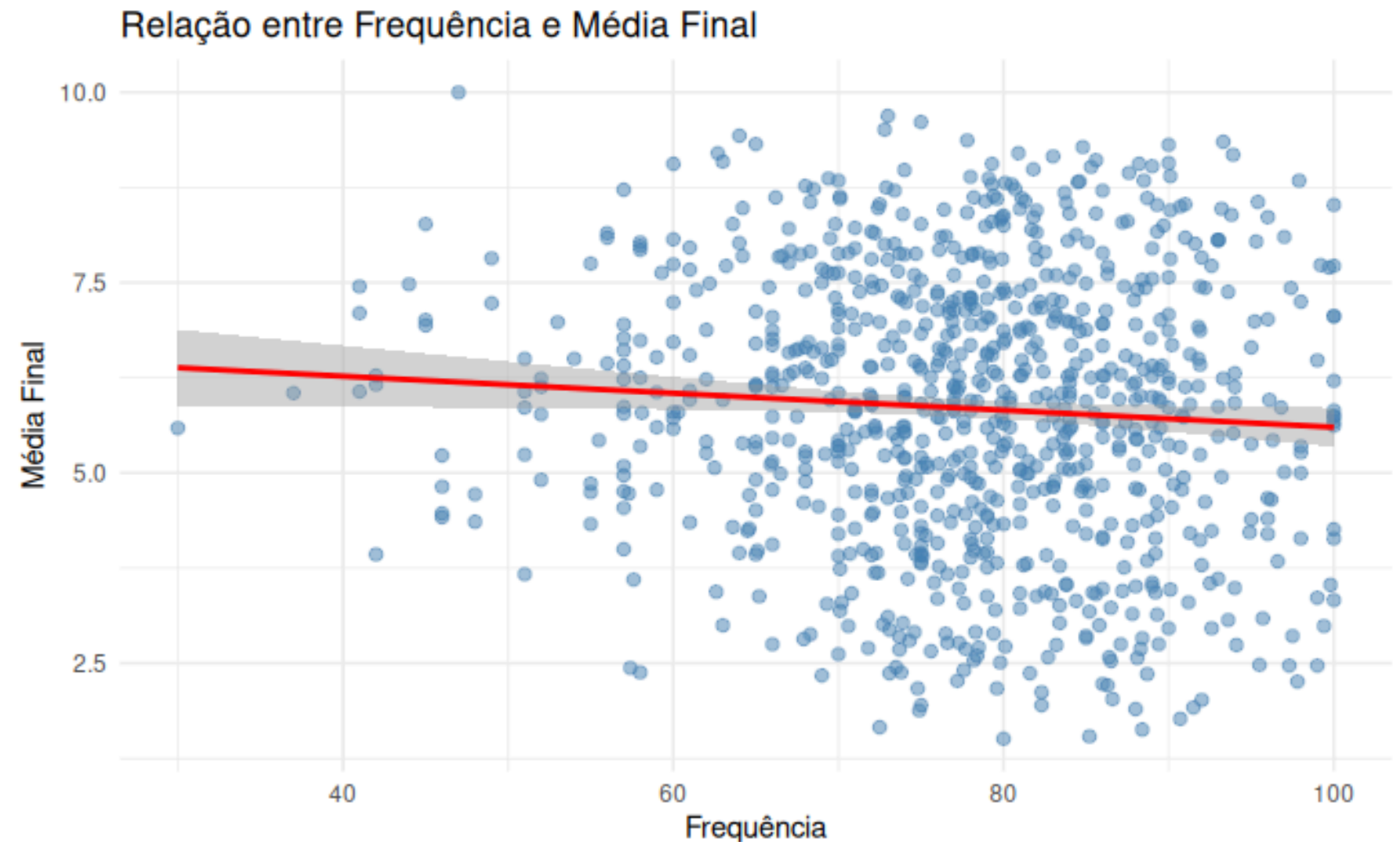
# Conectando ao servidor
con <- dbConnect(
  drv,
  dbname = "postgres",
  host = "localhost",
  port = 5432,
  user = "postgres",
  password = "1234"
```

Obtenção dos Dados

```
matriculas <- dbReadTable(con, "participa_em")

matriculas$key <- paste(
  matriculas$nusp_aluno,
  matriculas$vigencia,
  matriculas$sigla_disciplina,
  matriculas$codigo_turma,
  sep = "_"
)
row.names(matriculas) <- matriculas$key

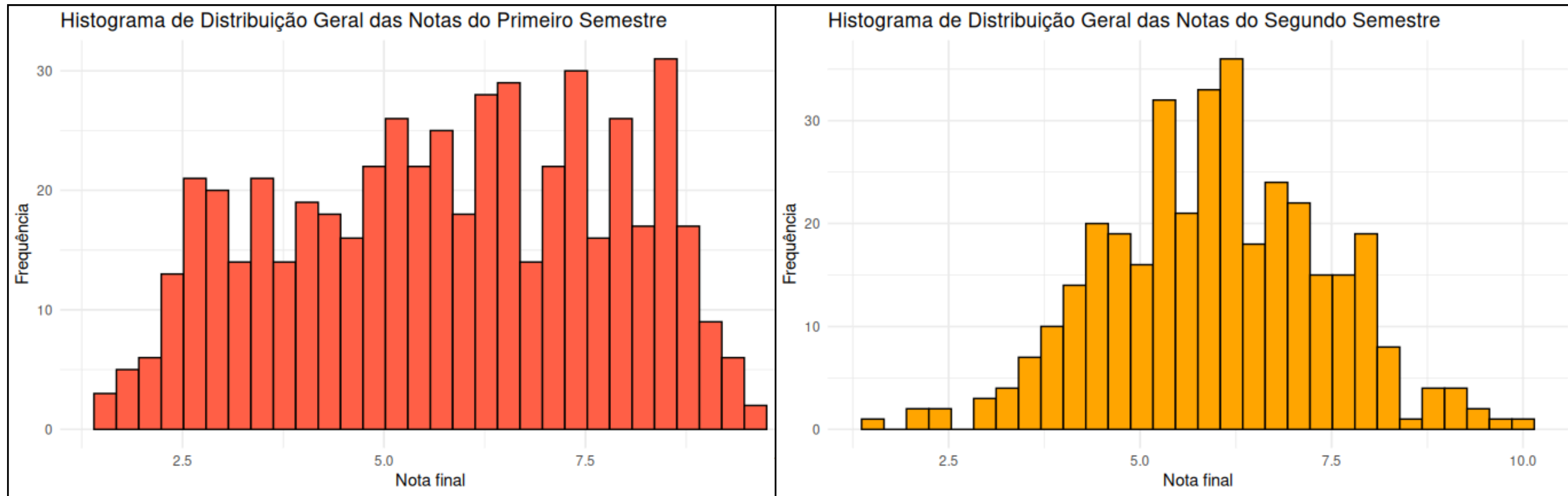
matriculas$key <- NULL # opcional: remove a coluna auxiliar
head(matriculas)
```



```
ggplot(df, aes(x = frequencia, y = media_final)) +
  geom_point(alpha = 0.5, color = "steelblue", size = 2) +
  geom_smooth(method = "lm", color = "red", se = TRUE) +
  labs(
    title = "Relação entre Frequência e Média Final",
    x = "Frequência",
    y = "Média Final"
  ) +
  theme_minimal()
```

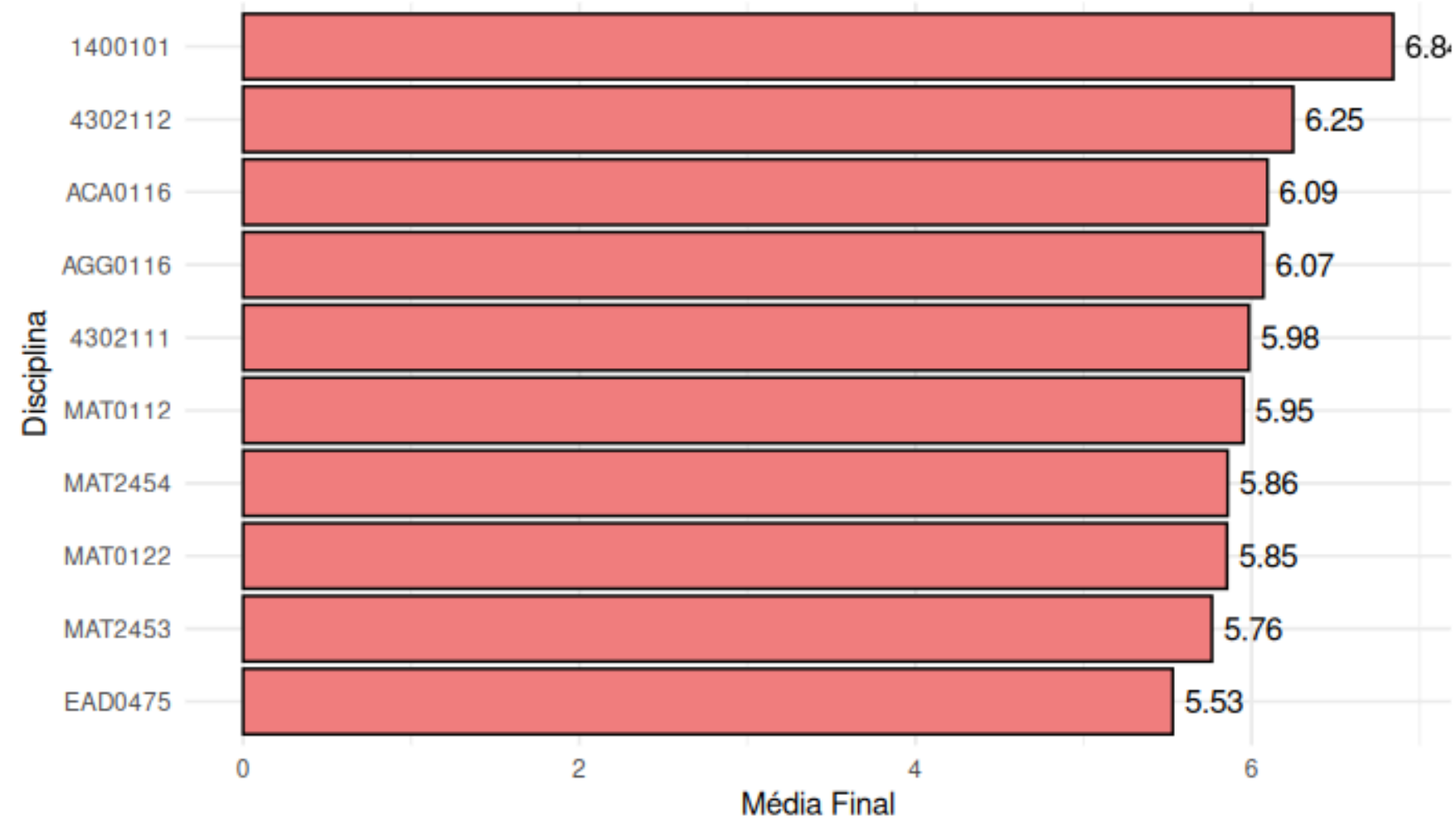
Análise Estatística dos Dados

Comparação entre semestres



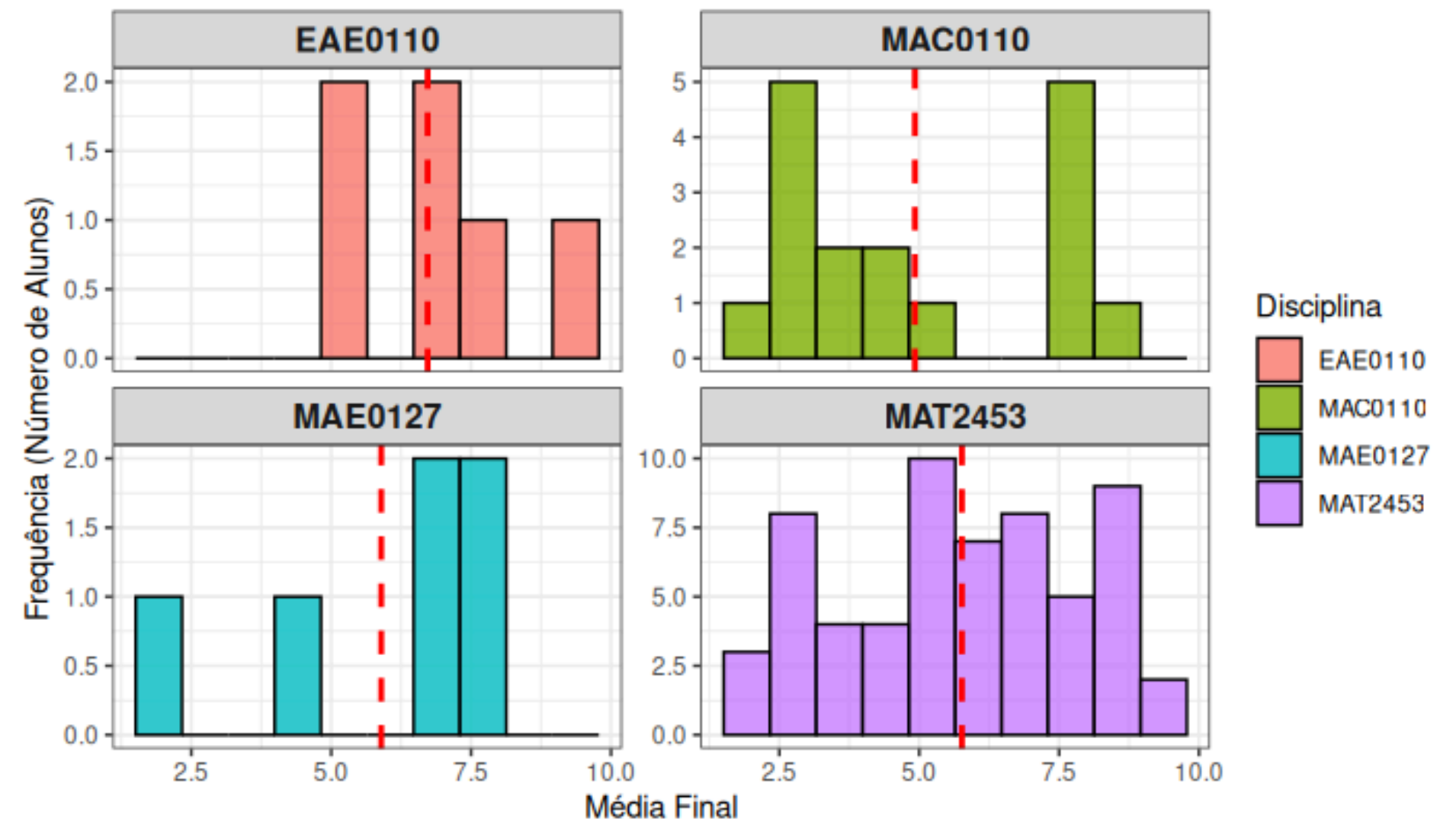
Análise Estatística dos Dados

Média de Notas por Disciplina (Top 10 com mais alunos)



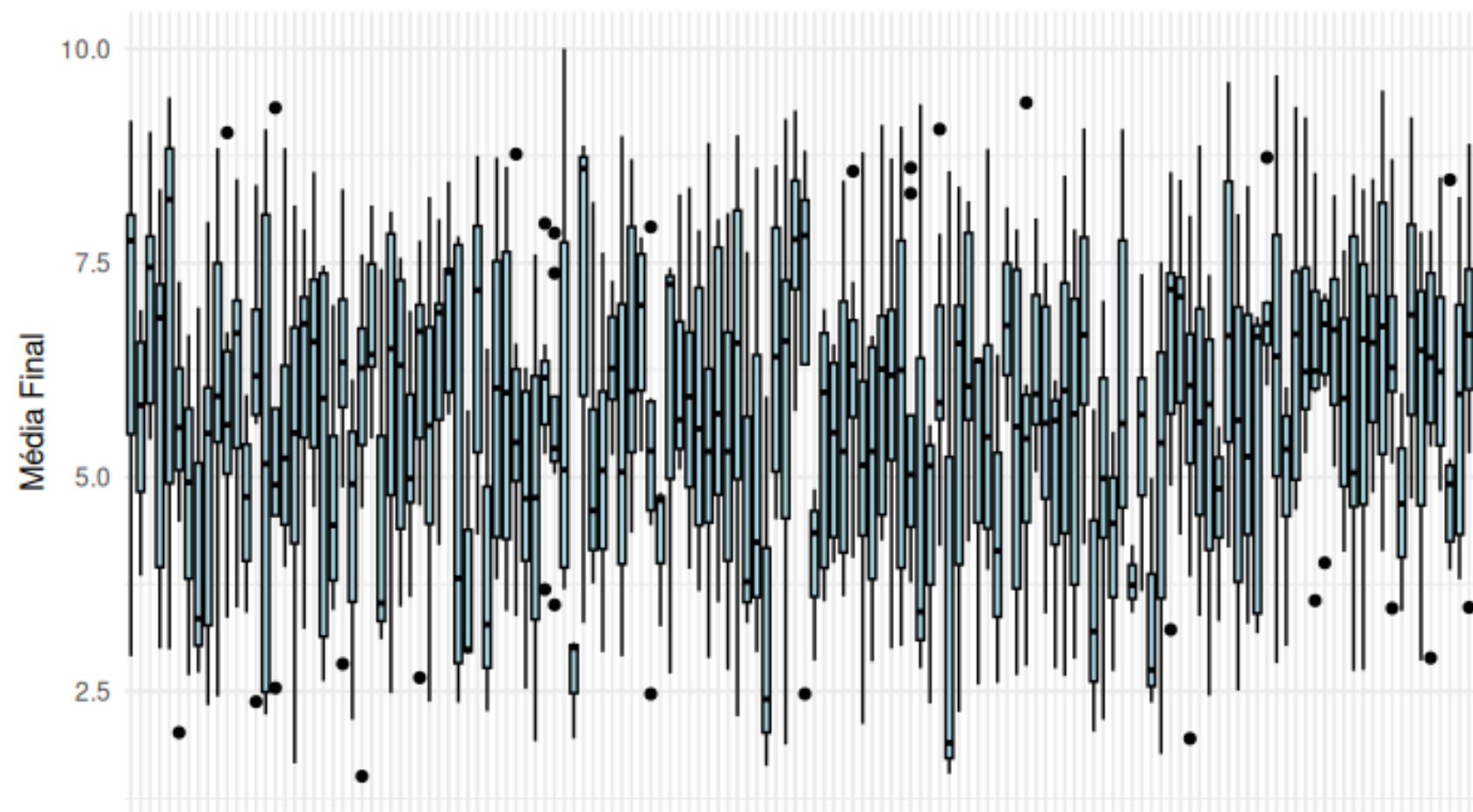
```
df %>%  
  count(sigla_disciplina) %>%  
  arrange(desc(n)) %>%  
  head(10) %>%  
  ggplot(aes(x = reorder(sigla_disciplina, n), y = n)) +  
  geom_col(fill = "skyblue", color = "black") +  
  geom_text(aes(label = n), hjust = -0.2, size = 4) +  
  coord_flip() +  
  labs(  
    title = "Top 10 Disciplinas com Mais Matrículas",  
    x = "Disciplina",  
    y = "Quantidade de Matrículas"  
  ) +  
  theme_minimal()
```

Distribuição das Notas (Média Final) por Disciplina



Análise Estatística dos Dados

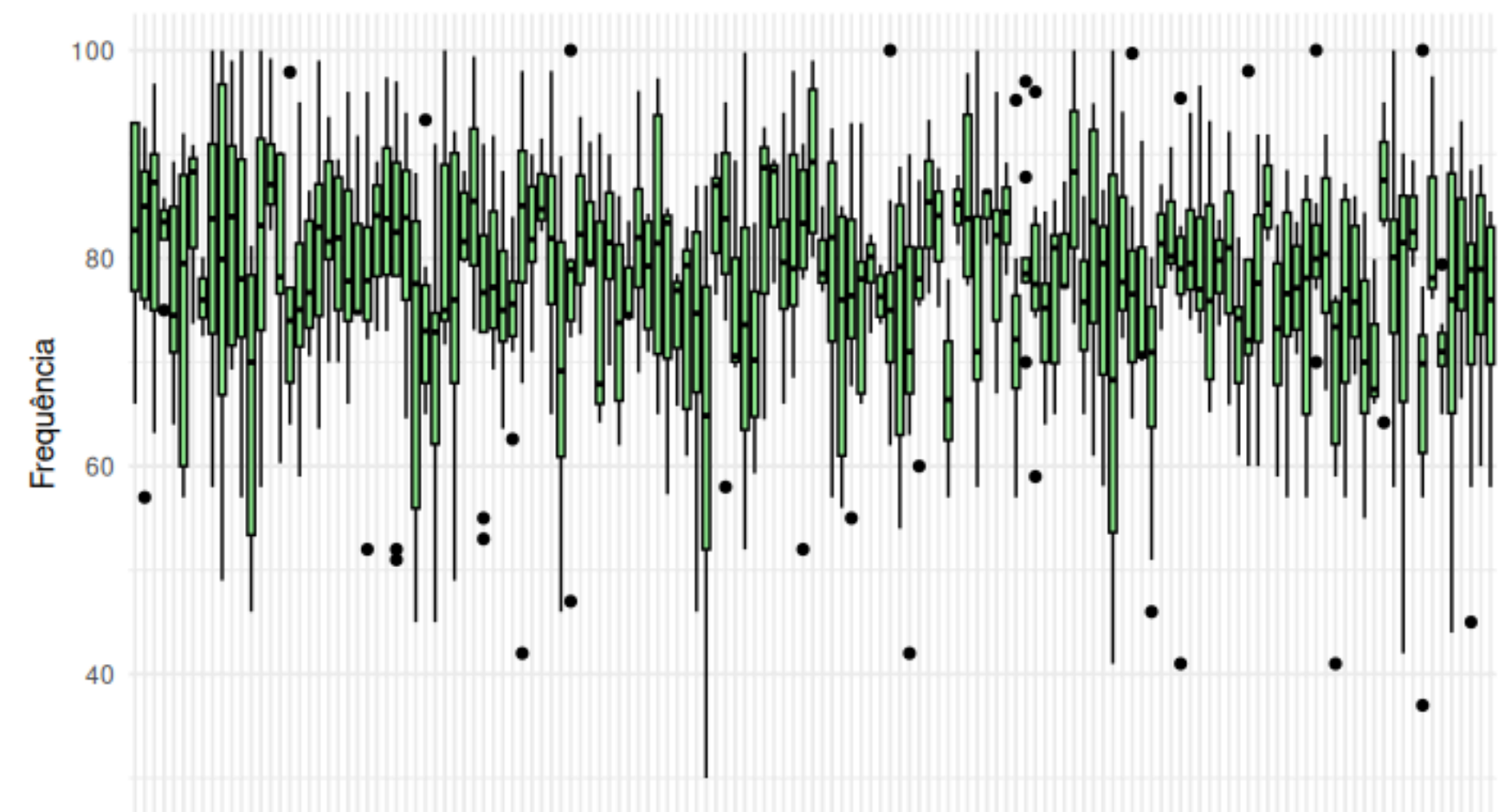
Distribuição das Notas por Aluno



```
df %>%
  group_by(nusp_aluno) %>%
  filter(n() >= 2) %>% # Apenas alunos com 2 ou mais disciplinas
  ungroup() %>%
  ggplot(aes(x = factor(nusp_aluno), y = media_final)) +
  geom_boxplot(fill = "lightblue", color = "black") +
  labs(
    title = "Distribuição das Notas por Aluno",
    x = "", # O título do eixo X já está vazio
    y = "Média Final"
  ) +
  theme_minimal() +
  theme(
    # A linha abaixo foi removida/comentada: theme(axis.text.x = element_text(angle = 45, hjust = 1))
    axis.text.x = element_blank(), # **Remove o texto (rótulos) do eixo X**
    axis.ticks.x = element_blank() # Remove as pequenas marcas (ticks) do eixo X
  )
```

```
df %>%
  group_by(nusp_aluno) %>%
  filter(n() >= 2) %>% # Apenas alunos com 2 ou mais disciplinas
  ungroup() %>%
  ggplot(aes(x = factor(nusp_aluno), y = frequencia)) +
  geom_boxplot(fill = "lightgreen", color = "black") +
  labs(
    title = "Distribuição da Frequência por Aluno",
    x = "", # O título do eixo X já está vazio
    y = "Frequência"
  ) +
  theme_minimal() +
  theme(
    # A linha abaixo foi removida/comentada: theme(axis.text.x = element_text(angle = 45, hjust = 1))
    axis.text.x = element_blank(), # **Remove o texto (rótulos) do eixo X**
    axis.ticks.x = element_blank() # Remove as pequenas marcas (ticks) do eixo X
  )
```

Distribuição da Frequência por Aluno



Análise Estatística dos Dados

Obtenção dos Dados

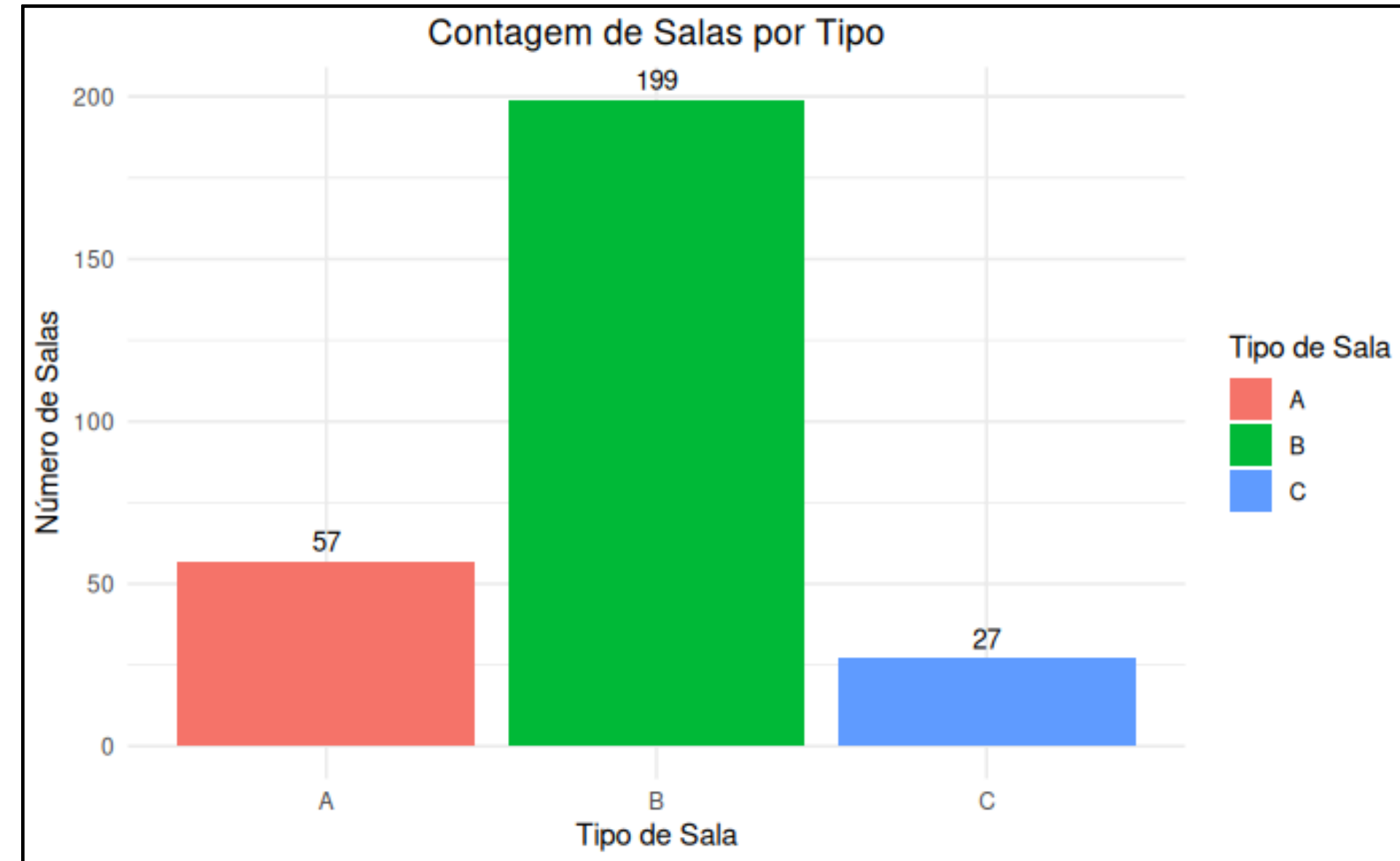
```
salas <- dbReadTable(con, "sala")
institutos <- dbReadTable(con, "instituto")

salas$key <- paste(
  salas$id_sala,
  salas$id_instituto,
  salas$lot_max,
  sep = "_"
)
row.names(salas) <- salas$key
```

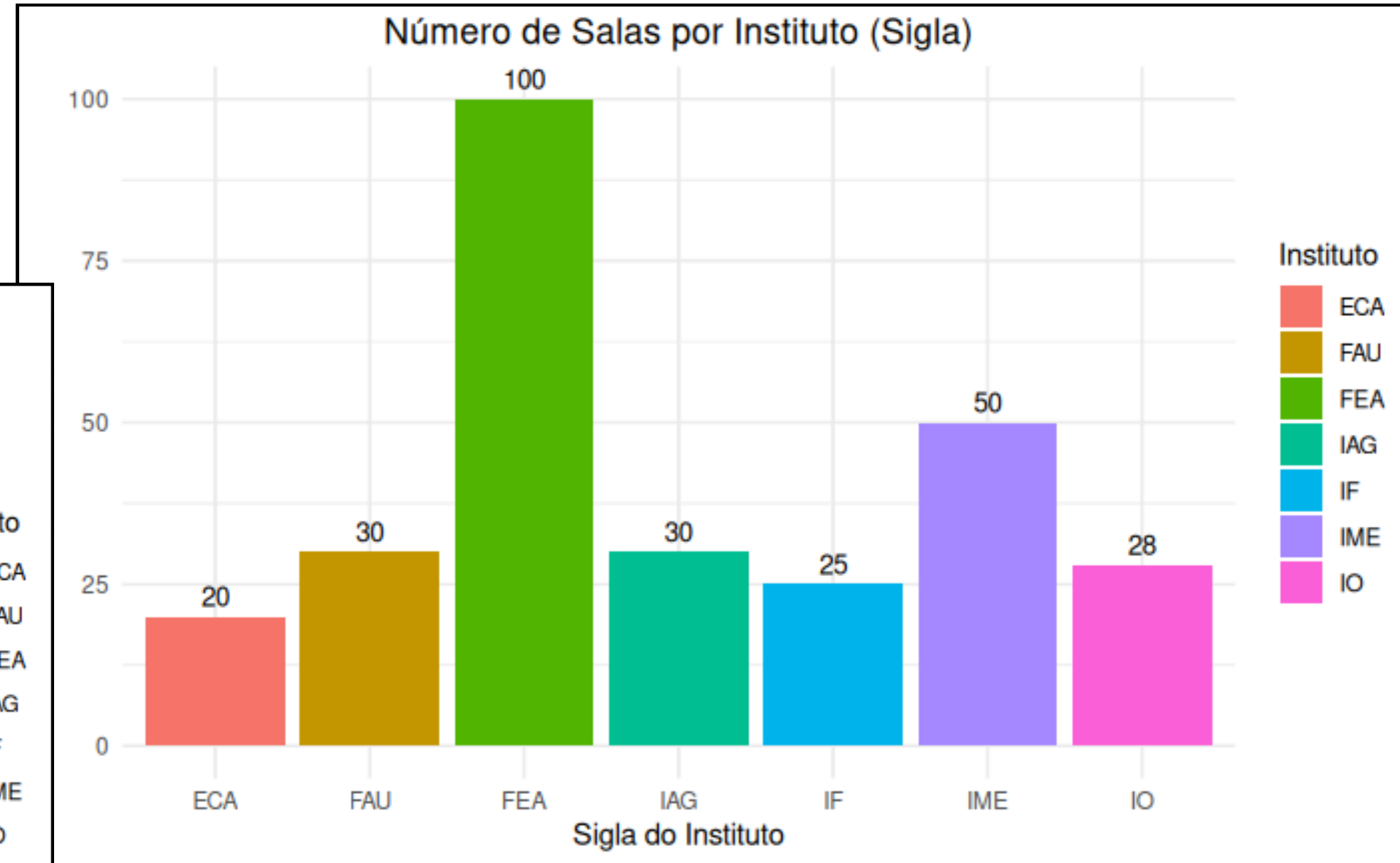
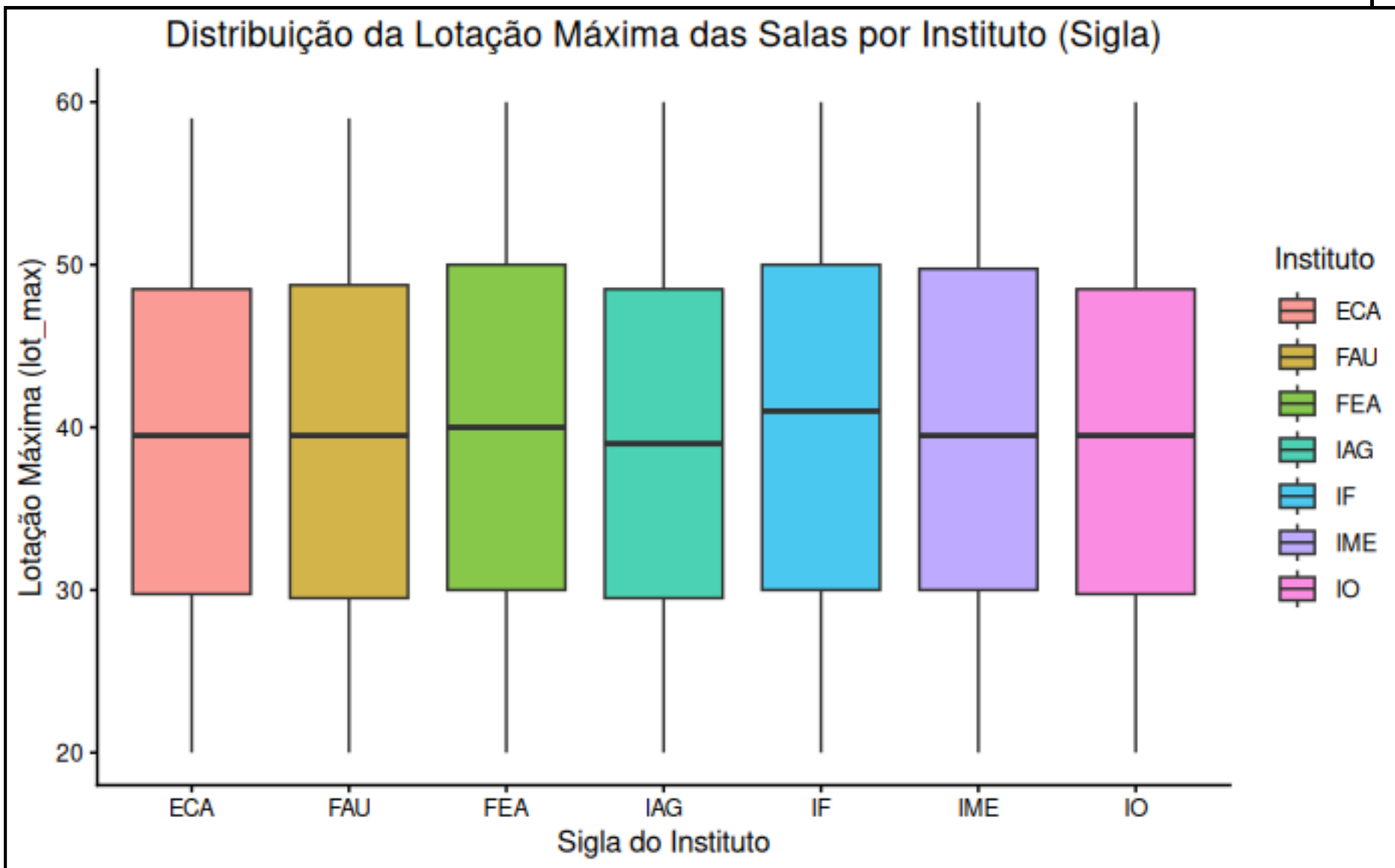
Código do gráfico

```
salas %>%
  ggplot(aes(x = tipo_sala, fill = tipo_sala)) +
  geom_bar() +
  labs(
    title = "Contagem de Salas por Tipo",
    x = "Tipo de Sala",
    y = "Número de Salas",
    fill = "Tipo de Sala"
  ) +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5)) +
  # Adiciona os valores no topo de cada barra
  geom_text(
    stat = 'count',
    aes(label = after_stat(count)),
    vjust = -0.5,
    color = "black",
    size = 3.5
  )
```

Análise de Dados de Salas



Análise Estatística dos Dados

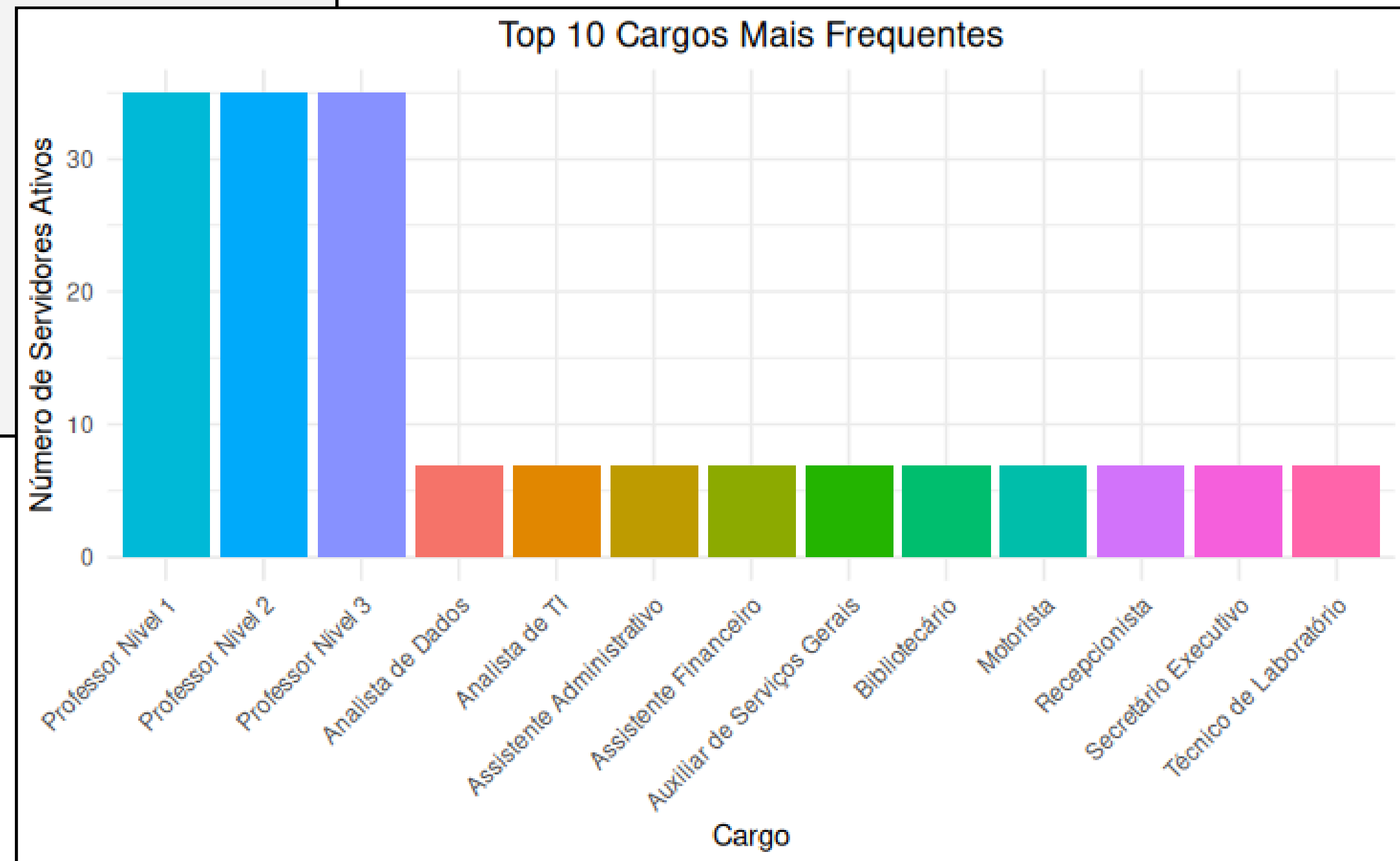


Análise Estatística dos Dados

Análise de Dados de Trabalho

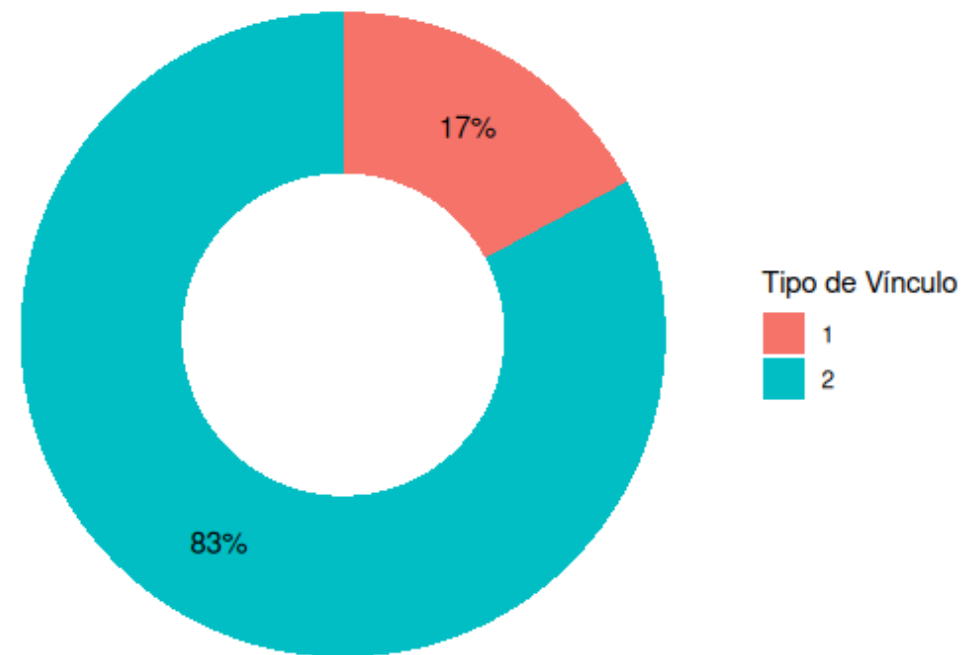
Código do gráfico

```
dados_ativos %>%  
  filter(cargo %in% top_cargos$cargo) %>%  
  ggplot(aes(x = reorder(cargo, cargo, function(x)-length(x)), fill = cargo)) +  
  geom_bar() +  
  labs(  
    title = "Top 10 Cargos Mais Frequentes",  
    x = "Cargo",  
    y = "Número de Servidores Ativos"  
  ) +  
  theme_minimal() +  
  theme(  
    axis.text.x = element_text(angle = 45, hjust = 1),  
    legend.position = "none",  
    plot.title = element_text(hjust = 0.5)  
  )
```

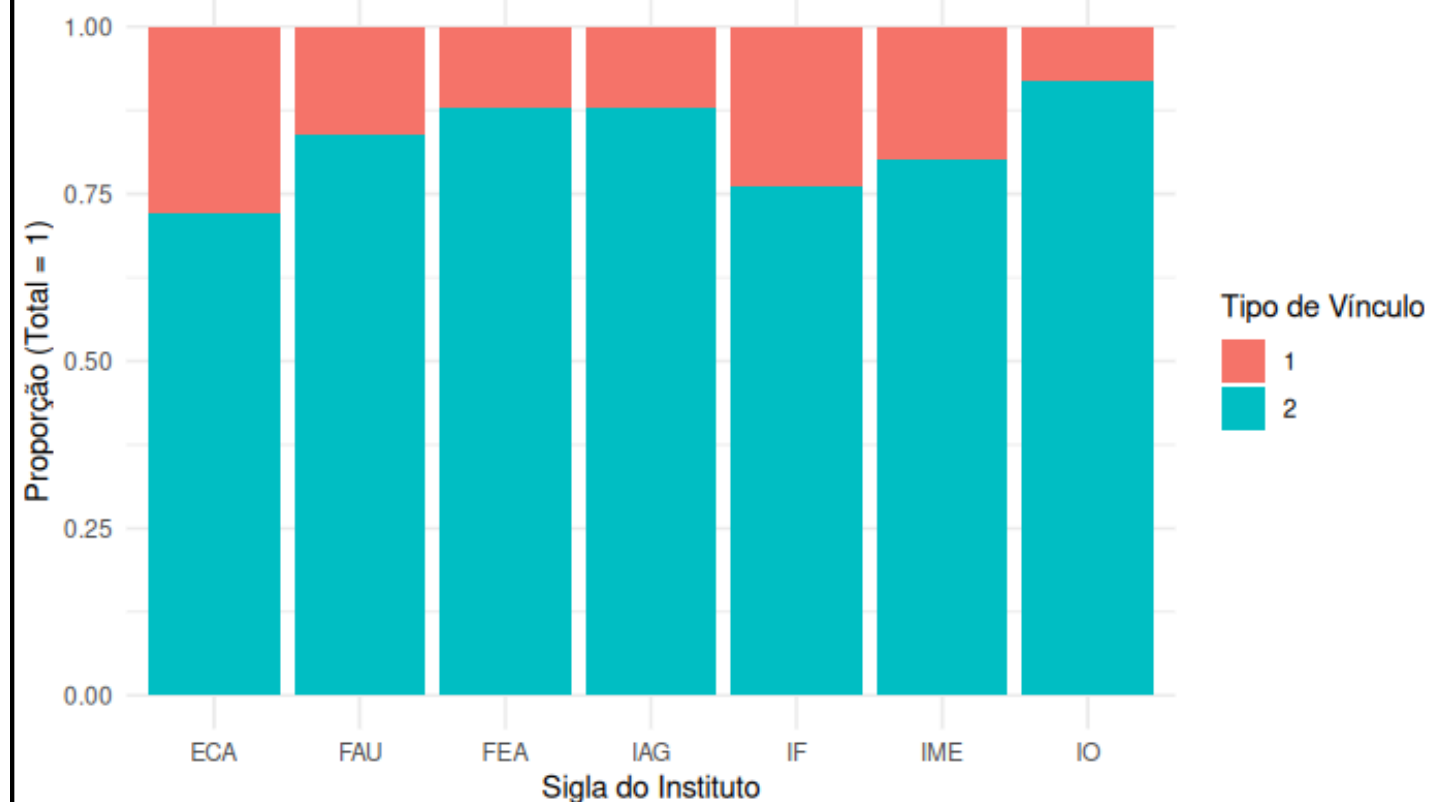


Análise Estatística dos Dados

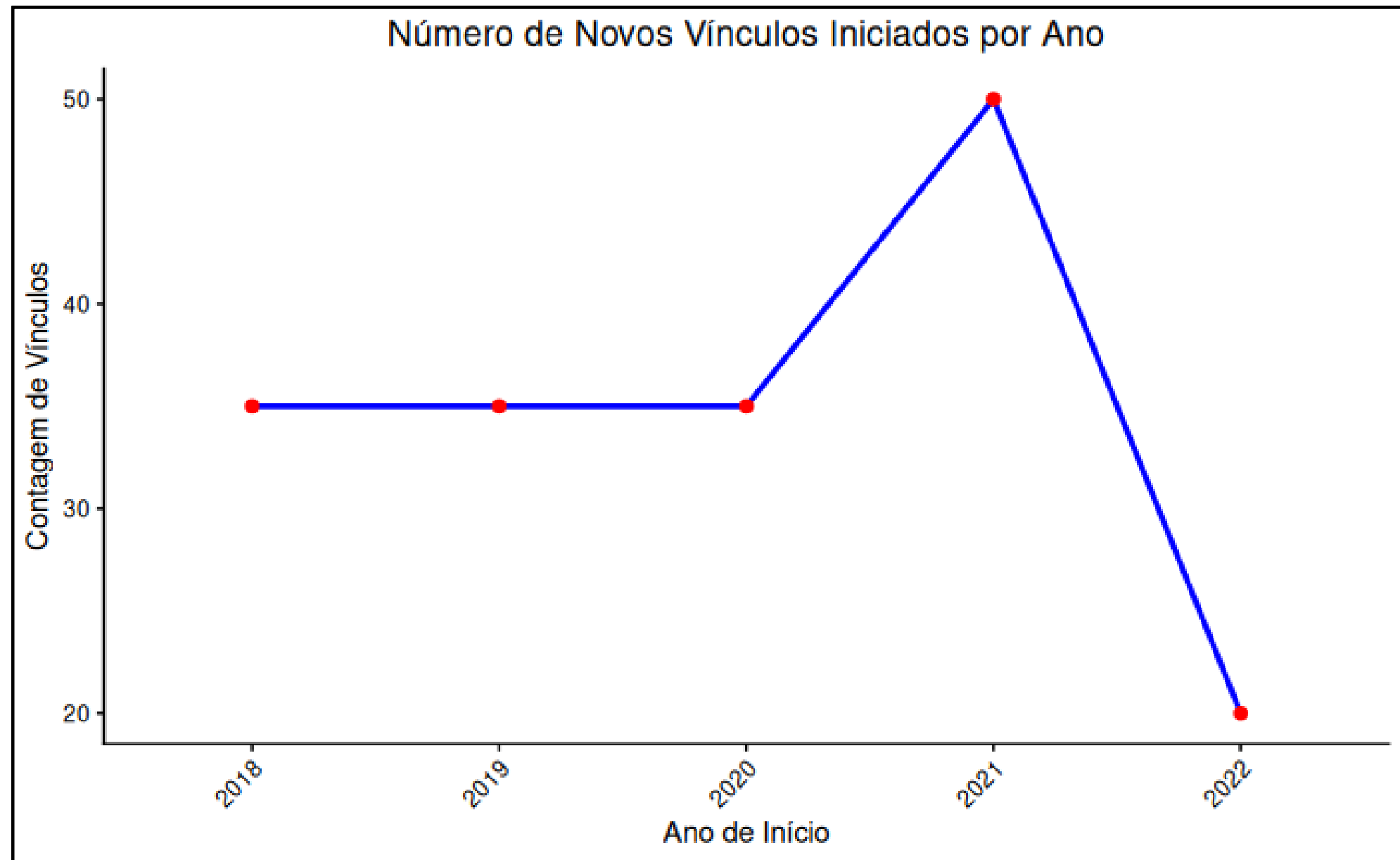
Distribuição de Servidores por Tipo de Vínculo



Proporção de Tipos de Vínculo por Instituto

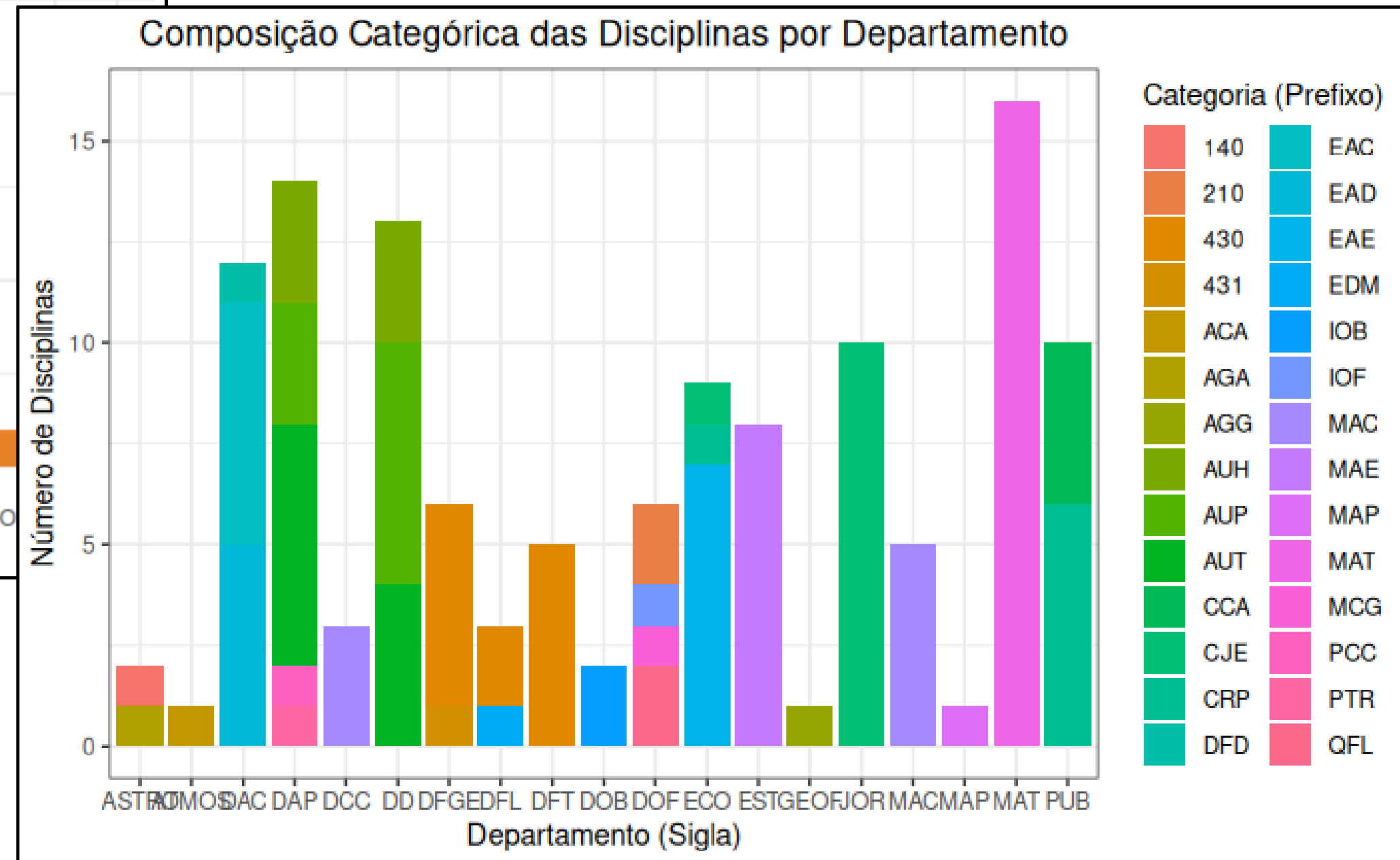
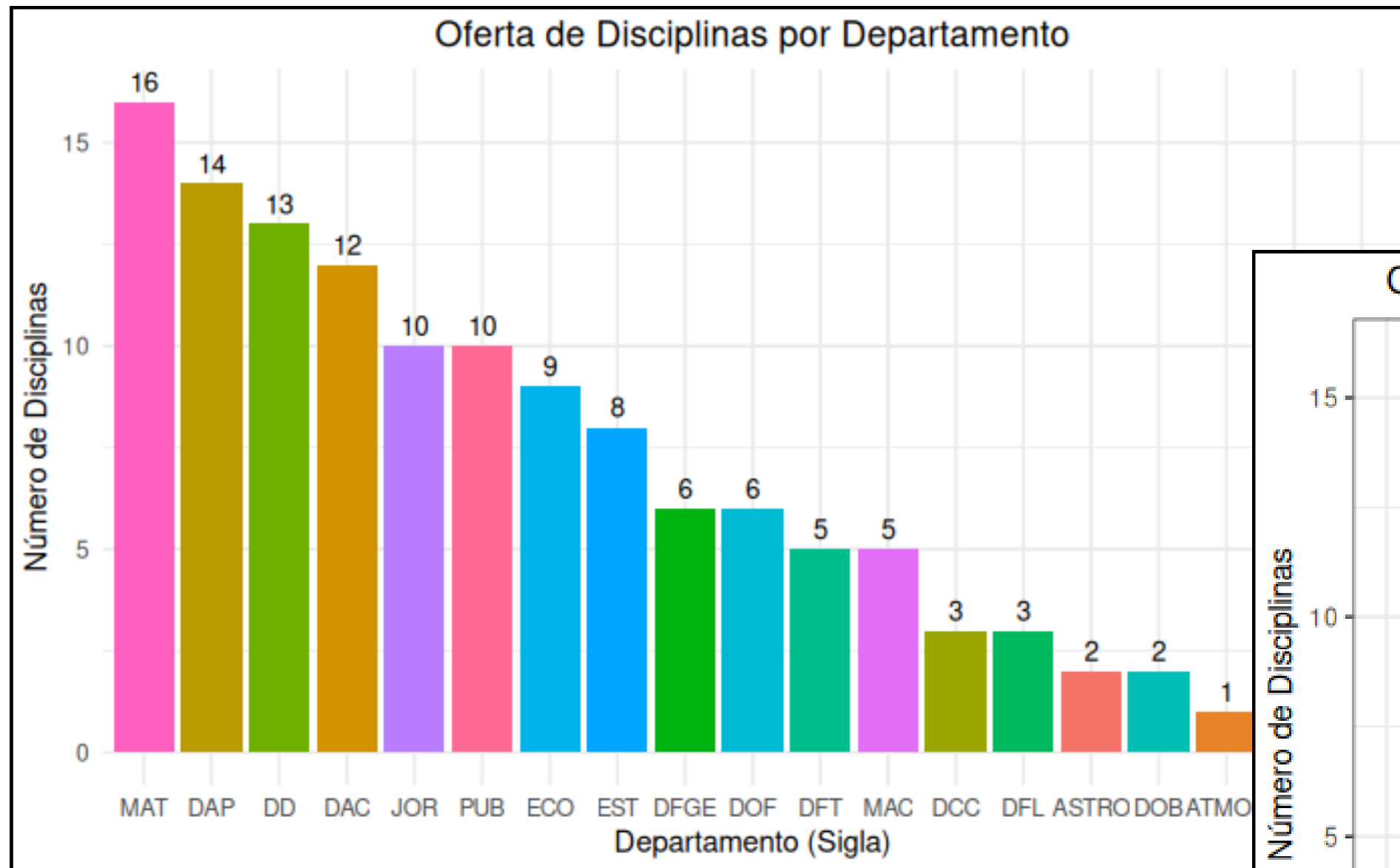


Número de Novos Vínculos Iniciados por Ano



Análise Estatística dos Dados

Análise de Dados de Disciplina



U-SPACE



Felipe Pinho
Gabriel Sanches
Kadu Barrille
Marcos O. Salvador

